



UNMESH MASHRUWALA
Innovation Cell
IIT BOMBAY

UMIC SeDriCa Recruitment Assignment 2026–27

Candidate instructions

Please read these instructions before you begin.

Section A: General / Common Questions is compulsory for every candidate. Answer all questions in this section.

Section B: Subsystem-wise Assignment contains the role-specific work. Technical candidates should attempt questions from at least two technical subsystems. Business candidates should complete the mandatory questions in Media & Web, the Business subsystem.

Choose the questions that match the role you are applying for and follow the mandatory and optional labels shown within each subsystem.

Section A is assessed for clear, authentic and thoughtful answers grounded in your own observations and experiences. Be specific, and stay within each word limit; longer answers are not automatically stronger.

Section B is assessed on your understanding of the problem, the quality of your reasoning, and how carefully you support your work with research, evidence and results.

Keep brief notes on your approach, decisions and anything you could not complete. Include these notes with your submission so we can understand how you worked through the assignment.

Submission guidelines

Prepare a report using this assignment document. Add your personal details and answer the questions you are attempting directly below each question. Remove questions you are not attempting, and make sure your answers and figures are easy to read.

Submit a GitHub repository containing your report, code, images, data and any other files needed to review or run your work. Make sure the recruitment team can access it.



UNMESH MASHRUWALA
Innovation Cell
IIT BOMBAY

Typed answers are preferred. If a handwritten sketch or calculation is useful, include a clear image of it in your report.

Deadline: 4 October, 11:59 PM.

- List the references you used at the end of your report, grouped by question.
- You are not expected to have all the technical background before starting. Use the questions and their suggested resources to guide your research, and explain any assumptions you make.
- For questions about the assignment, use the query form shared by the recruitment team.

To learn more about UMIC SeDriCa, you can explore these channels:

Youtube: https://www.youtube.com/channel/UCkwi2H7zV2texcM6bQ_UagA

Facebook: <https://www.facebook.com/innovationcell/>

Instagram: https://instagram.com/umic_iitb

Linkedin:
<https://www.linkedin.com/company/unmesh-mashruwala-innovation-cell-iit-bombay>

For updates and questions, please join the recruitment WhatsApp group:

WhatsApp Group link | https://chat.whatsapp.com/L1XuzLRiInXlwHpDMZ7qNa?mode=ems_share_t



A note before you begin

What we are looking for

Show us how you approach an unfamiliar problem. Where a question asks for a concrete result, include it alongside a concise explanation of your reasoning. Use the platform details provided in the assignment, cite useful research, and note key decisions or anything you could not complete.

Using AI tools

AI tools are welcome as part of your process. Check their suggestions against the assignment details and make sure you understand and can explain the work you submit.

If you used AI substantially, briefly describe how in your appendix and include the prompts that shaped your work. This helps us understand your process.

Keep the full chat history of your AI conversations, including every prompt you used. You will be asked to share it in the submission link.

Personal Information

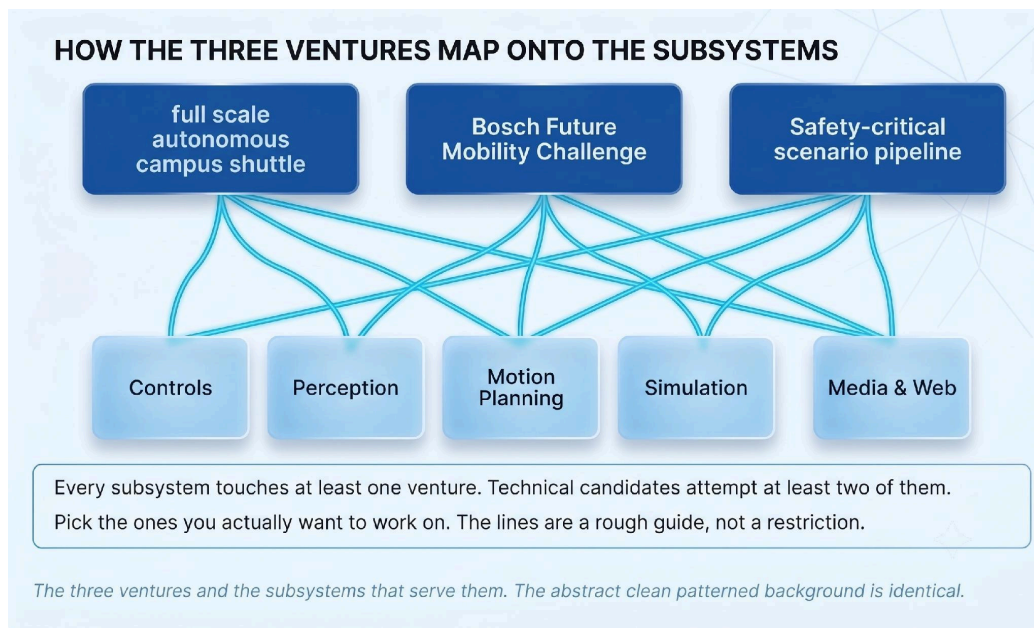
Name	Tanish Vishal Jain
Roll number	25B4244
Department	ESED
GitHub repository link	https://github.com/tanishjain0911-beep/UMIC-SeDriCa-Assignment_25B4244

#	Subsystem	Questions attempted
1	Perception	1,2
2	Motion Planning	1
3		



What UMIC SeDriCa is building in 2026–27

Read this section before attempting anything. Most questions below refer back to it.



Venture 1: Full scale autonomous campus shuttle (buggy)

We are converting a six-seater electric buggy into an autonomous campus shuttle for IIT Bombay's internal roads. It will share those roads with pedestrians, cyclists and other vehicles. Because it carries people, our design decisions must account for safety as well as performance.

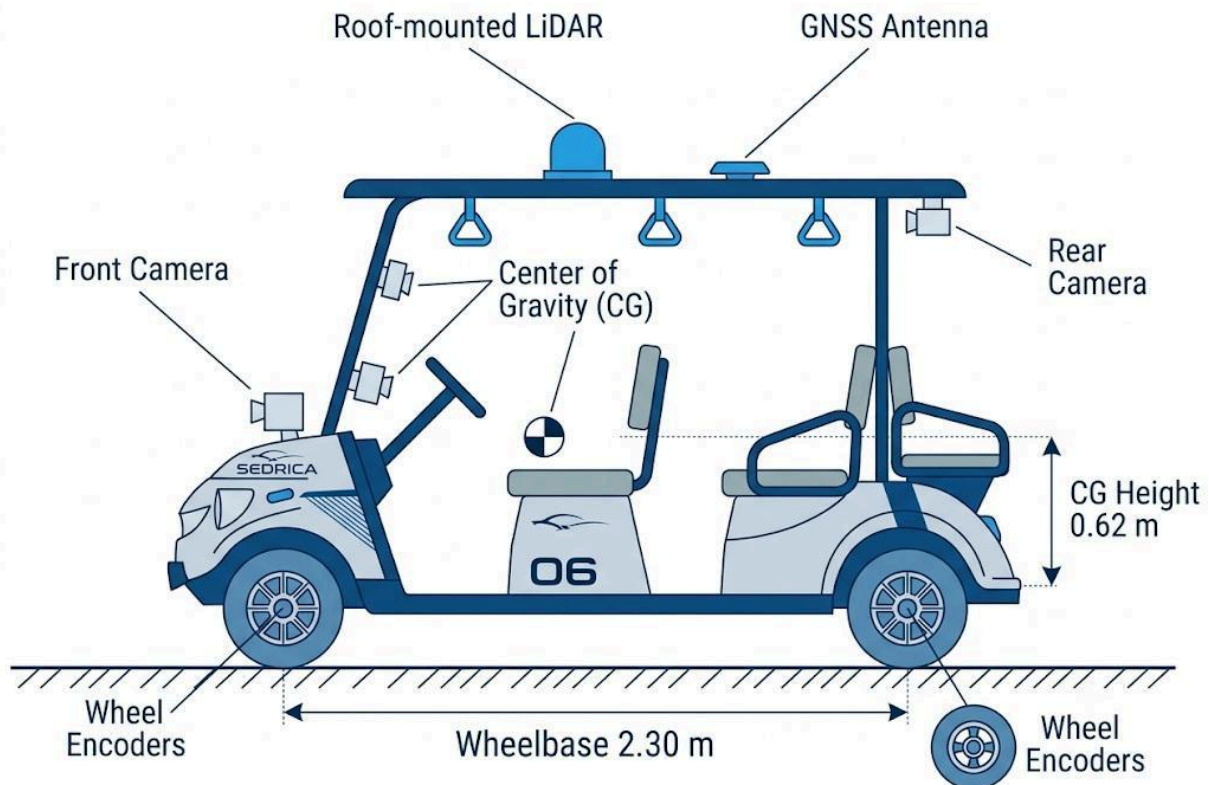
Working platform specification. Use the figures below unless a question says otherwise. If a parameter is missing, state and justify your assumption.

Parameter	Value
Configuration	6-seater electric buggy, rear-wheel drive
Kerb mass	550 kg
Max payload	400 kg (6 occupants + compute + sensors)
Wheelbase	2.30 m
Track width	1.10 m



Parameter	Value
CG height (laden, estimated)	0.62 m
Tyre rolling radius	0.28 m
Traction motor	48 V DC, 5 kW peak, via speed controller with 0-5 V throttle input
Battery	48 V, 150 Ah
Steering	Mechanical rack & pinion, ratio $\approx 18:1$, 3.5 turns lock-to-lock, max road-wheel angle $\pm 32^\circ$
Brakes	Rear drum brakes, cable-actuated from a single pedal
Top speed (stock)	24 km/h
Operating speed cap	20 km/h
Planned compute	NVIDIA Jetson Orin NX (perception/planning) + STM32 (low-level control)
Planned sensors	1 $\times$ 32-beam LiDAR, 2 $\times$ RGB cameras, GNSS-RTK + 9-DOF IMU, wheel encoders on both rear wheels

Campus operating conditions: Main Gate to Research Park and back on IIT Bombay's internal roads; daytime, dry weather and speeds up to 20 *km/h*; mixed traffic; a safety driver aboard with a mechanical override.



Buggy working platform. Use these figures unless a question says otherwise.

Venture 2: Bosch Future Mobility Challenge (BFMC)

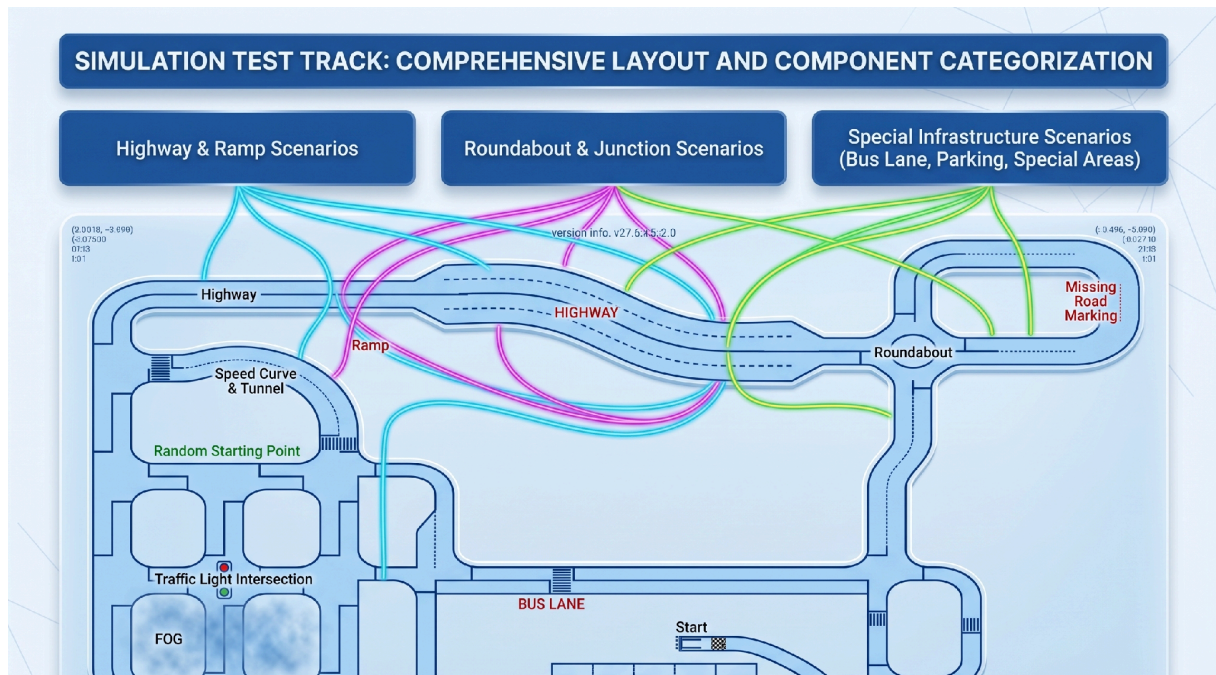
The Bosch Future Mobility Challenge (BFMC) is an international student competition in Cluj-Napoca, Romania. Teams develop autonomous driving software for a provided 1:10 scale vehicle. The official platform uses a Raspberry Pi, a Nucleo controller, a camera and an IMU. Its track includes intersections, traffic lights, crossings, parking, a roundabout, a highway section and a tunnel. The vehicle can also use competition V2X services, including traffic-light messages and localization data.

Teams are assessed through autonomous driving on the track, compliance with traffic rules and a jury presentation. Consult the current official regulations for the competition format and rules.

Official BFMC resources:

Regulations: <https://regulation.boschfuturemobility.com/>

Technical documentation: <https://documentation.boschfuturemobility.com/>



Conceptual overview of the BFMC scale-city track and the elements the car must handle. For exact geometry, use the official track maps on the BFMC documentation's [Race Track page](#).

Venture 3: Safety critical scenario generation pipeline

You cannot validate a self-driving policy by driving it around and hoping something interesting happens. Dangerous events are rare: that is the entire problem. A vehicle would have to drive hundreds of millions of kilometres to accumulate enough near-misses to say anything statistically meaningful about its safety.

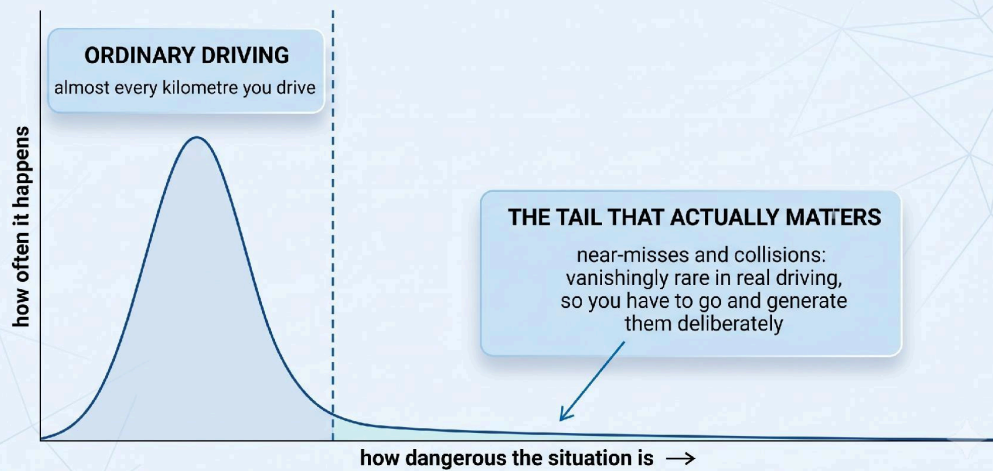
This project for Mercedes-Benz Research and Development searches for traffic situations that expose weaknesses in a driving policy. It scores how dangerous each situation is, filters out physically implausible cases and returns reproducible scenarios that can be used to examine the campus shuttle before it carries passengers.

SUGGESTED READING:

- Ding et al., *A Survey on Safety-Critical Driving Scenario Generation: A Methodological Perspective*, IEEE T-ITS 2023



WHY YOU CANNOT VALIDATE A DRIVING POLICY BY SIMPLY DRIVING IT AROUND



Dangerous events sit in the tail, which is why they have to be generated rather than waited for.

SEDARICA



Section A: General / Common Questions

Answer all questions in this section. It is compulsory for every candidate.

Q A.1: What are your short-term and long-term goals? (150 words or less)

Short term goals:

- Develop critical thinking and problem solving skills , in technical and software oriented domains, from projects, learning new algorithms, etc.
- Explore areas such as AI, ML, DS, not just theoretically from lectures, but from hands-on projects.

Long term goals:

- By the end of 4th year,, have a clearer idea of the kind of problems I want to work on and the type of role that suits me, whether that is research, technical development, or any type of career I choose.
- Develop a strong diverse skill set combining my departmental knowledge with AI, data science and computational methods, so that I can apply these tools to problems beyond purely software-based applications.
- Take up projects where I have ownership of the problem and the solution, rather than limiting myself to executing clearly defined tasks given by someone else.

Q A.2: What motivates you to join UMIC SeDriCa? (150 words or less)

After completing my CS101 course, I realised that I have a genuine interest in algorithms and data science-related fields, which led me to take the DS minor (DS203) in my third semester. Although it is a minor course, I found myself spending hours solving the Python modules and assignments purely out of interest and curiosity.

I deliberately chose to attempt this assignment after completing it, as I wanted to use it as a personal test of my motivation and interest in this field. I completed the assignment over two days, with a much higher-than-usual level of focus and attention. I found myself particularly



engaged with the **Perception** subdomain, where the preliminary image-processing knowledge I had gained through DS203 gave me a starting point to explore the problem further. Rather than simply arriving at a working solution, I was motivated to understand the problem more deeply, experiment with different approaches, and look for what I felt was the optimised solution.

Q A.3: A campus driving scenario *(250 words or less)*

Think of one specific situation you have seen on the IIT Bombay campus that would be difficult for the autonomous buggy. Name the location and describe what happens.

- a.** What makes this situation difficult? Give the place, people or vehicles involved, and the relevant conditions.

Location : Between **Chaayos T-point to H-17**

Description: The conditions of the **road** near H-17, is **very bad** due to construction activity going there. Due to daily movement of cement-carrying vehicles near that area, the road has become **very uneven**, and also **muddy and sticky**.

- b.** Would it challenge sensing, prediction, planning, control or the buggy's operating conditions? Explain briefly.

These conditions can create problem for the Control of the buggy, also for motion planning of the buggy. A route generated by Hybrid A may pass through a region with poor surface conditions , but it cant be directly classified as a static obstacle, due to the already narrow road and unpredicted movements of pedestrians and cycles.

- c.** Suggest one practical way the team could handle it.

A practical solution would be to incorporate **road-surface condition as a cost** rather than a static obstacle. Camera data could help identify poor road conditions, allowing Hybrid A* to prefer safer surfaces when possible. Where avoidance is impossible, the controller could reduce the buggy's speed while crossing the affected section.



TECHNICAL SUBDIVISION

Section B: Subsystem-wise Assignment

General note for all technical subsystems. Supporting your answers with well-crafted diagrams, plots and real numbers is heavily rewarded. Every subsystem below has mandatory question(s) and *optional/bonus* ones. Doing the mandatory work thoroughly beats doing all of them shallowly. Attempt bonuses only if the mandatory work is genuinely complete.

CONTROLS

Attempt Q1 (mandatory). Q2 is optional. Q3 is a bonus. Budget about 10 to 14 focused hours for Q1.

New tools worth knowing about: classical control (PID, Adaptive Pure Pursuit, Stanley) is still what ships in production vehicles at low speed and it is what we need first. But learned control (where a neural network, often trained by reinforcement learning, replaces or augments the controller) has moved from research curiosity to something companies are now cautiously deploying and Q3 below asks you to look into it.

Question 1: Vehicle Modeling, Path Tracking and Occupant Safety *(Mandatory)*

This question guides you through deriving planar vehicle motion models, implementing a lateral path-tracking controller and evaluating safety constraints for unrestrained passengers. Consider a 550 kg vehicle with a maximum speed of 20 km/h. The vehicle is neither a high-performance racecar nor a simple point-mass model. Before designing a controller, the vehicle's motion must first be modelled using its physical parameters. The controller must then be designed to account for the vehicle's operating constraints and the safety of its occupants.



Part A: Planar Vehicle Modeling and Dynamic Model Transition

- a. Read up on the kinematic bicycle model for a car-like vehicle. Derive it for the buggy in the rear-axle frame, using the wheelbase from the platform spec. State every variable and every assumption and explain in your own words *why* this is the standard starting model for a low-speed vehicle.
- b. Read up on the dynamic bicycle model, which adds lateral tyre forces. Write it down with a linear tyre model. You will not find a published cornering stiffness for a golf cart tyre: estimate one from typical passenger-car tyre ranges scaled by the per-tyre vertical load you can compute from the platform spec and defend the estimate in a few lines.
- c. In your own words (No need of a simulation): at what point does the difference between these two models start to matter for a vehicle that never exceeds 20 *km/h*? What is the argument, physically, for why a low-speed shuttle can often get away with the simpler model when a race car cannot?

Part B: Control it

- d. Implement one path-tracking controller (Pure Pursuit or Stanley, your choice) using the kinematic model from (a), on a simple hand-built reference path (a straight section followed by one curve is sufficient; you do not need to trace a real road).
- e. This vehicle carries seated passengers with no seatbelts and a 0.62 m CG. Research how lateral acceleration limits are set for passenger comfort and rollover avoidance in low-speed vehicles, calculate the same for the buggy from that reasoning, show the calculations and check whether your controller's commands stay within it on your test path at 20 *km/h*.
- f. Tune your controller's one main parameter (lookahead distance or cross-track gain, depending upon your controller selection) and show, with a plot, how tracking performance changes as you vary it. Pick a value and justify it.

DELIVERABLES

Derivations with every assumption stated, commented simulation code, tracking plots, the *comfort/rollover* calculation, decision log, failure log.

Question 2: PID vs. MPC for a vehicle that must not be trusted blindly (Optional)

You do not need to simulate anything for this question: it is a formulation and reasoning exercise and it rewards depth of understanding over volume of code.



- a. Consider the buggy approaching a pedestrian who may or may not step into the road (see the Motion Planning subsystem for understanding the decision-making side of this). Design a PID controller for longitudinal speed tracking that would work for this vehicle. Write pseudocode, explain the role of each term and state what happens physically when you increase each gain too far: connect this explicitly to a vehicle carrying unsecured passengers, not just to overshoot on a graph.
- b. Now formulate the same problem as a Model Predictive Control (MPC) problem for joint longitudinal and lateral control of the buggy. Define the cost function, the horizon and the constraints: including, explicitly, a constraint that encodes "do not exceed the *comfort/rollover* limit you derived in Q1". You do not need to solve it; showing that you have set up a correct, well-motivated optimisation problem is the point.
- c. For each controller, describe in two or three sentences a specific scenario where it would fail on this vehicle, and why.
- d. Research Model Predictive Control vs. PID in the context of safety-critical control and summarise, in your own words, the practical argument for why a team building a passenger-carrying vehicle would eventually want to move to MPC even though PID is far simpler to get running first.

DELIVERABLES

The PID pseudocode, the MPC formulation written out fully (cost, constraints, decision variables), the failure-scenario discussion and your summary of the PID-vs-MPC argument.

Question 3: Can a learned controller be trusted here? (Bonus)

Reinforcement-learning-trained controllers are increasingly used for vehicle control tasks that classical control finds awkward and increasingly discussed for production use, not just research. This question does not ask you to train one; it asks you to think clearly about whether you should.

- a. Find and briefly summarise one peer-reviewed paper or technical article published in 2023 or later describing an RL-trained or deep network based learned controller for a real or simulated ground vehicle.
- b. What did the learned controller do better than a classical baseline in that work and what did it do worse or not address at all?
- c. The buggy carries passengers with no seatbelts. Would you trust a learned controller to directly command steering or throttle on this vehicle today? Give your actual position, not a



UNMESH MASHRUWALA
Innovation Cell
IIT BOMBAY

hedge and justify it. Specifically, address how you would validate a learned controller's safety before it ever carries a person. This connects directly to the Simulation subsystem's scenario-generation pipeline, so feel free to reference it.

SEDRICA



PERCEPTION

Questions 1 and 2 are mandatory. Questions 3 and 4 are bonus. Start with a working baseline for each mandatory question before trying an optional extension. The bonus questions are for candidates who want to go deeper; they do not replace the mandatory work.

A laptop with Python, NumPy, Matplotlib, and OpenCV is enough for Questions 1 and 2. Question 3 also uses a pretrained semantic-segmentation model and its inference library. You do not need a robot, ROS, a GPU, or model training. Budget about 12 to 16 focused hours for Questions 1 and 2 together, and another 5 to 7 hours for each bonus question you attempt. These estimates include reading and testing but not the report for your other subsystem.

Starter data: [SeDriCa_perception_starter_data.zip](#). The archive contains inputs for Questions 1, 2, and 4 and is small enough to run offline after download. These scenes are synthetic teaching cases, not competition recordings. Files ending in `_reference.csv` are evaluation labels; prediction code must not read them. In Question 4, `reference_scans.csv` is an input map, not an evaluation label. Question 3 uses the separately linked Cityscapes8 images and a pretrained model.

Submit your code or notebooks, exact run instructions in `Perception/README.md`, and the requested figures and tables in your report. Label axes and units, record thresholds and random seeds, and explain at least one failure. Follow the appendix instructions for references, decision and failure logs, and AI use.

Question 1: The road disappears (Mandatory)

Imagine the BFMC car following a painted lane. On a clear stretch, both boundaries are easy to see. A few frames later, a shadow covers part of the road, one boundary fades, and a bright seam offers a convincing but false line. The controller cannot steer from a description such as “the road seems to bend left.” It needs a lane centre at a near row and another at a farther row. Your job is to produce those two image coordinates for each frame, show when the estimate is unreliable, and find out which visual cues survive the difficult frames.

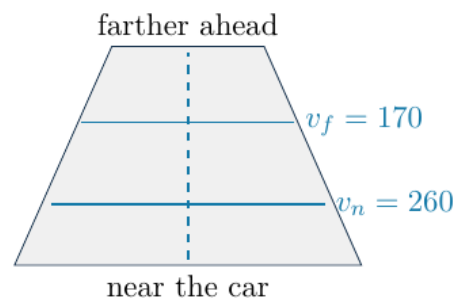
DATA AND COORDINATES

Use the three 24-frame sequences in `city/lane/clear`, `city/lane/shadow`, and `city/lane/missing`. Each frame is 480×320 pixels. The image origin is at the upper-left: u increases to the right and v increases downward. At row $v_n=260$, report the horizontal coordinate u_n of the lane centre. Do the



same at row $v_f=170$ to obtain u_f . The centre is halfway between the left and right road boundaries at that row, even when you can see only one boundary. This definition is about the road you intend the car to follow, not the brightest line in the image.

The file `city/lane_reference.csv` contains the true u_n and u_f for evaluation. It is not an input to your detector. The provided calibration files are not needed for this question. Work in image coordinates and concentrate on which pixels are useful evidence when the markings weaken.



The sketch is a coordinate guide. Measure on the actual frames.

YOUR WORK

- Look before coding.** Choose one frame from each sequence and mark the two boundary cues and the centre you would want the car to follow at v_n and v_f . If a boundary is hidden, mark where you think it continues and explain your evidence. Identify the shadow or seam that could fool a simple white-pixel threshold. This sketch is your first hypothesis, not a label to feed into the code.
- Make a small baseline.** Write a function that takes one frame and returns (u_n, u_f) , together with a way to say “unknown” for either coordinate. A colour threshold, edge map, line fit, or a combination is enough. State the image region, thresholds, and any assumption about lane width or shape. Run the unchanged baseline on all 72 frames.
- Interrogate the evidence.** For one difficult frame, display the stages of your baseline as a small panel: the original image, the region of interest, the cue it extracts, and the proposed lane centre. Put the same stages beside a clear frame. Point to the exact pixels that persuade your method, and to the shadow, seam, or missing boundary that persuades it wrongly. If you use a colour space or an edge operator, explain what it made easier to separate in these frames.
- Repair a real failure.** Choose a failure you observed, such as the shadow, missing boundary, or false seam. Revise your baseline using a cue you can justify. For example, use road width,



earlier frames, a restricted search region, or agreement between the near and far rows. If you claim a real-time method, use only the current and earlier frames. Return a confidence value in $[0,1]$ with each estimate, and state the evidence that raises or lowers it. This value is a ranking of trust; it need not be a calibrated probability.

- e. **Measure the change.** For each sequence and each row, compute mean absolute error in pixels on frames where your method returned a coordinate. Also report the fraction of frames marked unknown. Report both numbers for the baseline and your revision: a method cannot look accurate simply by refusing every hard frame. Use *city/lane_reference.csv* only after producing predictions. Show two failure frames, including one where a plausible bright line was wrong, and explain what the code did in each case.

The near and far centres need not move together. If your method gives a plausible near centre but an implausible far centre, explain what the car would infer incorrectly and how your confidence rule exposes the disagreement.

DELIVERABLES

An annotated frame from each sequence; a clear-versus-difficult evidence panel; a baseline-versus-revision error and unknown-rate table; a plot of estimated and reference u_n through the missing-line sequence; the confidence rule; and a short explanation of the change you made and what it fixed.

FURTHER EXPLORATION

Try a second approach such as temporal filtering or segmentation. If you want a geometry extension, use the supplied calibration points for a bird's-eye view. Compare on the same frames. Complexity is useful only if its benefit is visible.

READING

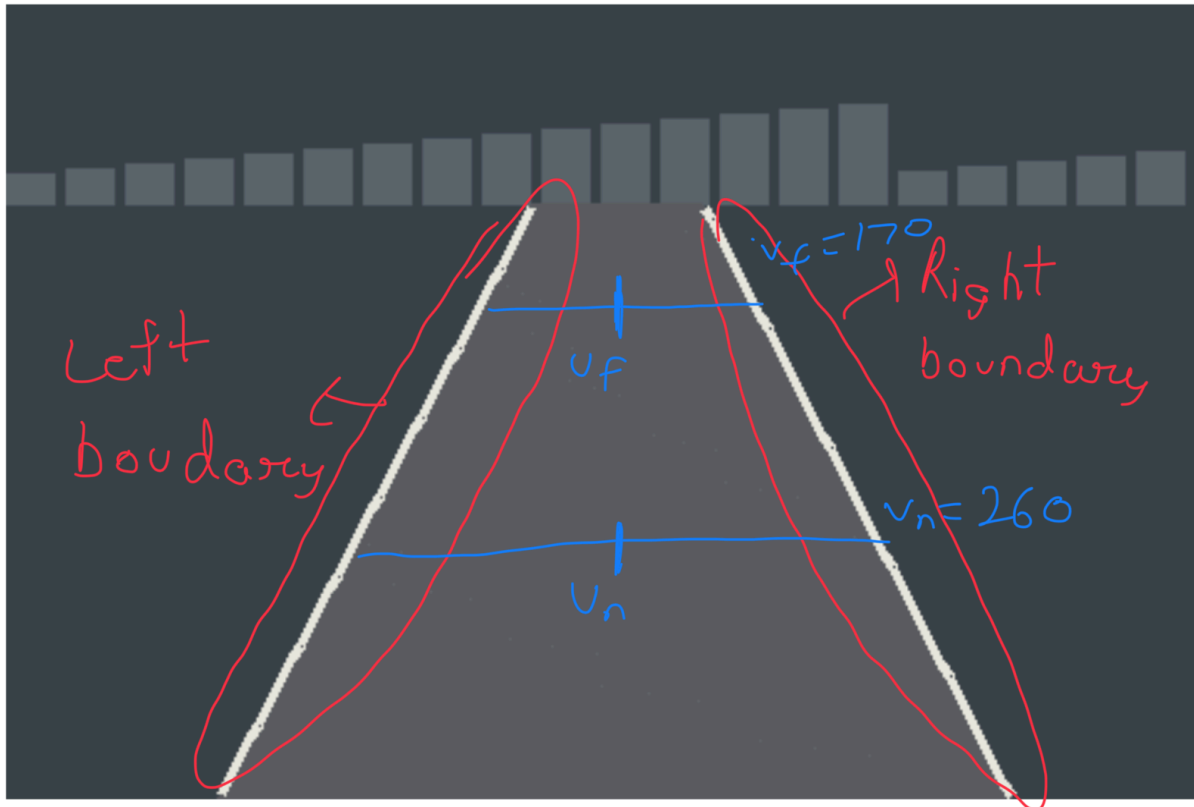
OpenCV's [color-space tutorial](#) and [edge-detection tutorial](#) are possible starting points. The [BFMC track description](#) shows the road elements motivating this question.

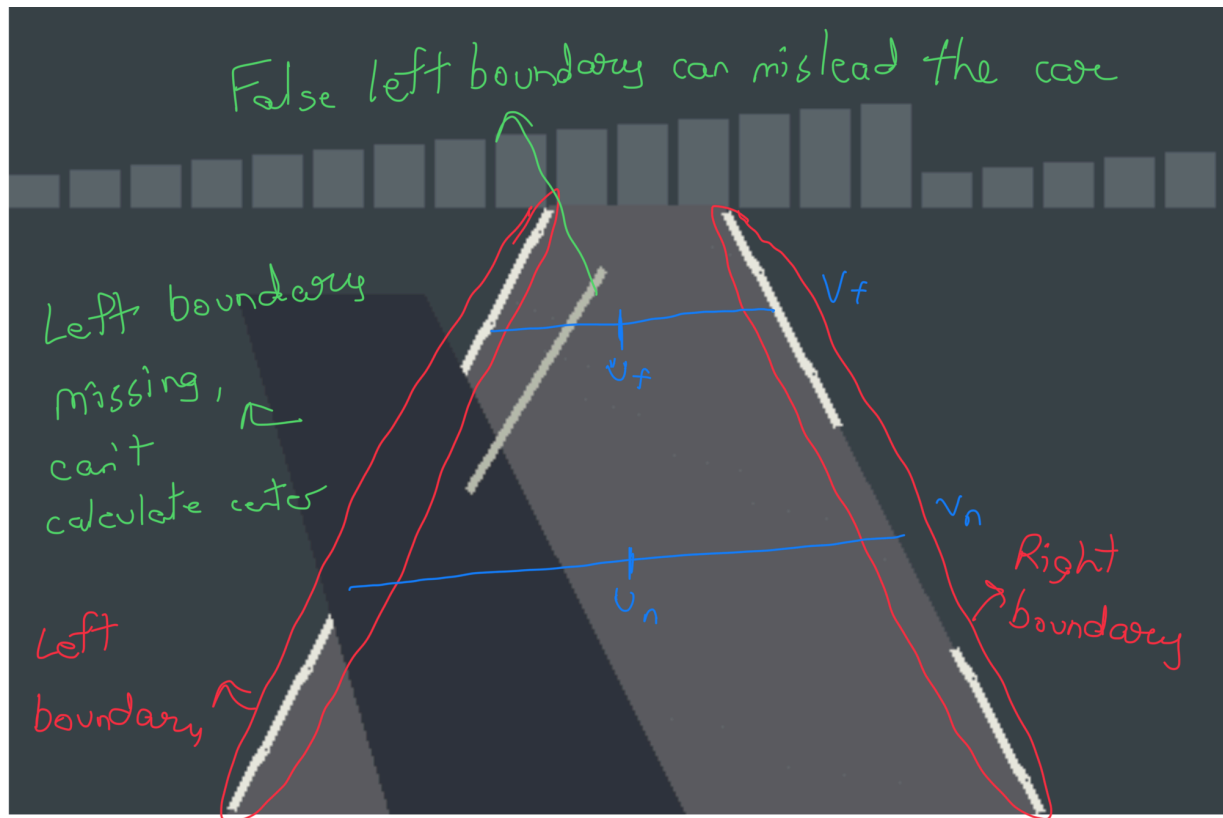
Answer:

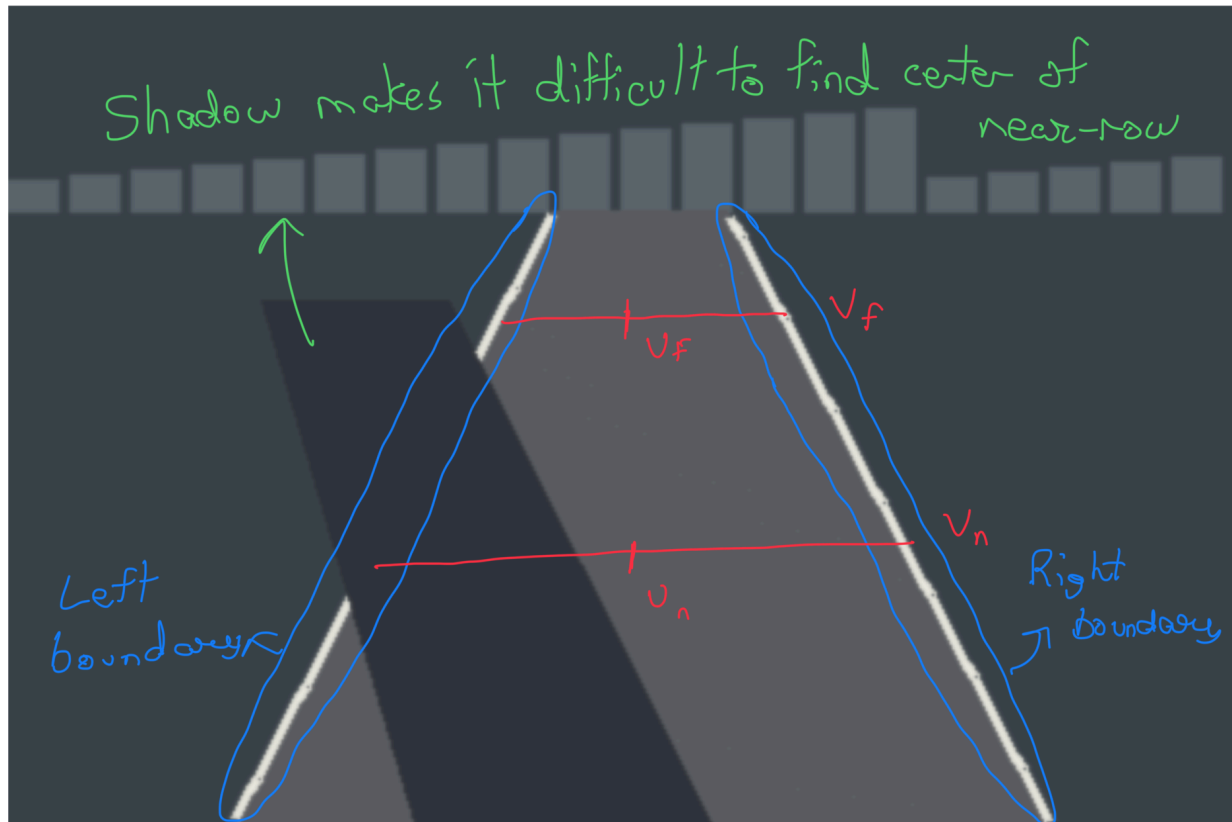
a)



UNMESH MASHRUWALA
Innovation Cell
IIT BOMBAY







The boundary where there is a shadow is predicted by interpolation .



b) Baseline Code:

https://drive.google.com/file/d/1_p2Qcf1RI6C6iDLKLEerpGWqnKemHGLc/view?usp=sharing

Baseline Method

For each frame, the baseline looks only at the two required rows, $v = 170$ and $v = 260$.

At each row, the image is divided into left ($u < 240$) and right ($u \geq 240$) halves. The **brightest pixel** in each half is selected as the corresponding lane-boundary candidate.

If the brightness of a candidate is below the chosen threshold, that boundary is marked **unknown**. Otherwise, the lane centre is calculated as the midpoint of the detected left and right boundaries:

$$u = \frac{u_L + u_R}{2}$$

Thus, the baseline makes an independent brightness-based estimate at each required row and **does not use information from other rows or regression**.

d)

Proposed Method

For each frame, the image is processed **row by row**. Pixel brightness is calculated as $R + G + B$, and each row is divided into left ($u < 240$) and right ($u \geq 240$) regions.

For each region, the **brightest pixels above the threshold** are identified. If multiple pixels have the same maximum brightness, their u -coordinates are averaged to obtain one



representative boundary coordinate for that row. Rows where no pixel crosses the threshold are treated as missing observations.

All valid boundary points are then used to fit **separate linear regression lines** for the left and right lane boundaries:

$$u_L = a_L v + b_L, u_R = a_R v + b_R$$

At the required rows $v = 170$ and $v = 260$, the method first checks the actual image data. If a boundary passes the threshold, its **actual detected coordinate** is used. If no valid pixel exists, the corresponding regression line is used to estimate the missing boundary.

The lane centre is then calculated as:

$$u = \frac{u_L + u_R}{2}$$

giving the **far centre at** $v = 170$ and **near centre at** $v = 260$.

Confidence Score

The confidence score reflects how much the final centre relies on actual image evidence:

- **Both boundaries actual:**

- $C = 1.0$

- **One actual, one regression:**

- $C = 0.75 - \frac{MSE_{regression}}{10}$



- Both regression:

- $$C = 0.50 - \frac{MSE_L + MSE_R}{10}$$

Here, the MSE measures how closely the detected boundary points follow their respective fitted regression lines. A smaller MSE therefore results in a smaller confidence penalty.

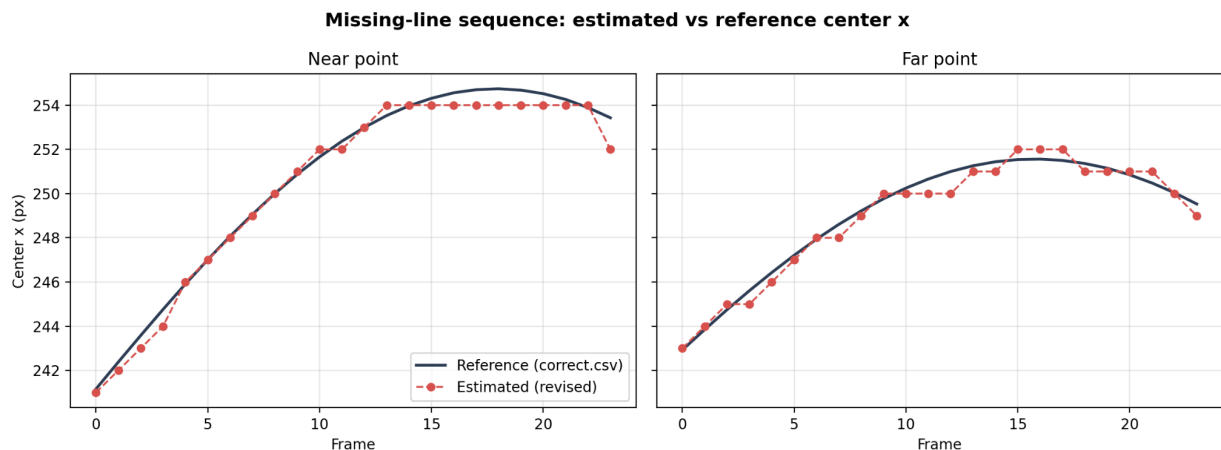
e)

Baseline results vs. correct.csv

Sequence	Point	Frames detected	Unknown frames	Unknown fraction	Mean abs. error	Max error	Within 2 px
clear	near	24/24	0/24	0.0%	1.54 px	1.91 px	24/24
clear	far	24/24	0/24	0.0%	1.58 px	1.94 px	24/24
missing	near	19/24	5/24	20.8%	1.58 px	1.91 px	19/19
missing	far	0/24	24/24	100.0%	n/a	n/a	n/a
shadow	near	18/24	6/24	25.0%	0.46 px	0.74 px	18/18
shadow	far	12/24	12/24	50.0%	0.47 px	1.00 px	12/12

Revised method results vs. correct.csv

Sequence	Point	Frames detected	Unknown frames	Unknown fraction	Mean abs. error	Max error	Within 1 px	Within 2 px
clear	near	24/24	0/24	0.0%	0.35 px	0.84 px	24/24	24/24
clear	far	24/24	0/24	0.0%	0.28 px	0.52 px	24/24	24/24
missing	near	24/24	0/24	0.0%	0.37 px	1.43 px	23/24	24/24
missing	far	24/24	0/24	0.0%	0.36 px	1.00 px	24/24	24/24
shadow	near	24/24	0/24	0.0%	0.49 px	1.43 px	22/24	24/24
shadow	far	24/24	0/24	0.0%	0.47 px	1.26 px	22/24	24/24



Question 2: A crossing with conflicting evidence *(Mandatory)*

The same city car is approaching a crossing. In one run the light stays green. In another, it turns red while the car is moving. In a third, a person walks into the crossing. The camera offers imperfect evidence, and a V2X message can still say “green” after the light has changed because the message describes an older moment. Build a small perception-to-decision system that recommends GO, SLOW, or STOP at each frame and records why. The interesting part is not finding one perfect threshold. It is deciding what the car should do when evidence is late, noisy, or contradictory.

DATA AND COORDINATES

Use the three 24-frame sequences in *city/crossing*: *clear_green*, *late_red*, and *person_crossing*. Join each image to its row in *city/crossing_evidence.csv* using episode and frame. The row gives *time_s*, *speed_mps*, *distance_to_line_m*, two noisy vision scores in [0,1], a *v2x_light* state, and *v2x_sample_time_s*. A high *red_score* favours a red light. A high *person_score* favours a visible person, who may or may not be in the car's path. Neither score is a calibrated probability or a command to stop. The file *city/crossing_reference.csv* contains the true light, whether a person is on the crossing, and whether STOP is required. Use it only to evaluate decisions, never as an input to them.

You may use the supplied scores as the baseline. You are welcome to replace one or both with an image measurement such as a colour cue, frame difference, or a small detector, but model training is not expected. Inspect the images even if you use the supplied scores: a number is easier to interpret when you can see the light and the person's position. You can reuse your



Question 1 road region to judge whether a person is in the car's path; explain the connection if you do.

YOUR WORK

- a. **Map what the car knows.** Annotate one frame from each episode. Draw a signal-flow diagram with the image or vision scores, V2X state and timestamp, current speed, distance to the stop line, and your decision. Explain which observations are direct, which are uncertain, and why a visible person outside the road need not require STOP. The diagram should make clear that the evaluation labels are not available to the running system.
- b. **Make the first rule.** Implement a baseline using the supplied scores and V2X messages. For each frame return one of GO, SLOW, or STOP and a short reason string. Define what each state means in your system: for example, SLOW may mean that the car should approach cautiously while the evidence is unresolved. Choose and record your thresholds, then run the same rule on all three episodes. Show a timeline of its decisions beside the inputs.
- c. **Account for old messages.** For each row compute message age as $time_s$ minus $v2x_sample_time_s$. Find a frame where the message's state is misleading if its age is ignored. Revise your rule so it handles stale V2X data and disagreement between the camera and the message. Say what the car does while it cannot resolve the disagreement. Keep the baseline and revised timelines so the effect of this change is visible.
- d. **Check whether STOP is timely.** Assume a total processing and actuation delay $\tau=0.15$ s and comfortable braking magnitude $a=0.8$ m/s². For a car travelling at v metres per second, the approximate stopping distance is

$$d_{stop} = v\tau + \frac{v^2}{2a}.$$

At the first STOP request in each episode, compare $distance_to_line_m$ with this distance and state whether comfortable stopping still appears possible. If there is no STOP request, say so. If SLOW comes first, explain which evidence caused the later change. A correct STOP classification that arrives too late is still a failure worth discussing.

- e. **Test uncertainty.** Use a stated random seed to perturb the scores or drop some frames or messages. Keep the perturbation rule the same for baseline and revision, and run at least 20 trials. Using `city/crossing_reference.csv` only for scoring, count missed-STOP frames and unnecessary-STOP frames. For each episode that requires STOP, report the time from the first required frame to the first STOP request, or "none" if there is no request. Compare the baseline with your revised rule, then show one false alarm and one dangerous delay in the original frames. Explain why they happened.



Rules, a state machine, a running score, or a probabilistic model are all acceptable. If you call a score a probability, say how you checked its meaning. Do not treat an old V2X state as current.

DELIVERABLES

A signal-flow diagram; one annotated frame per episode; baseline and revised decision timelines with message age; a stopping-distance calculation at the first STOP request; a 20-trial table of missed and unnecessary STOP frames and response delays; and a short explanation of a false alarm and a dangerous delay.

FURTHER EXPLORATION

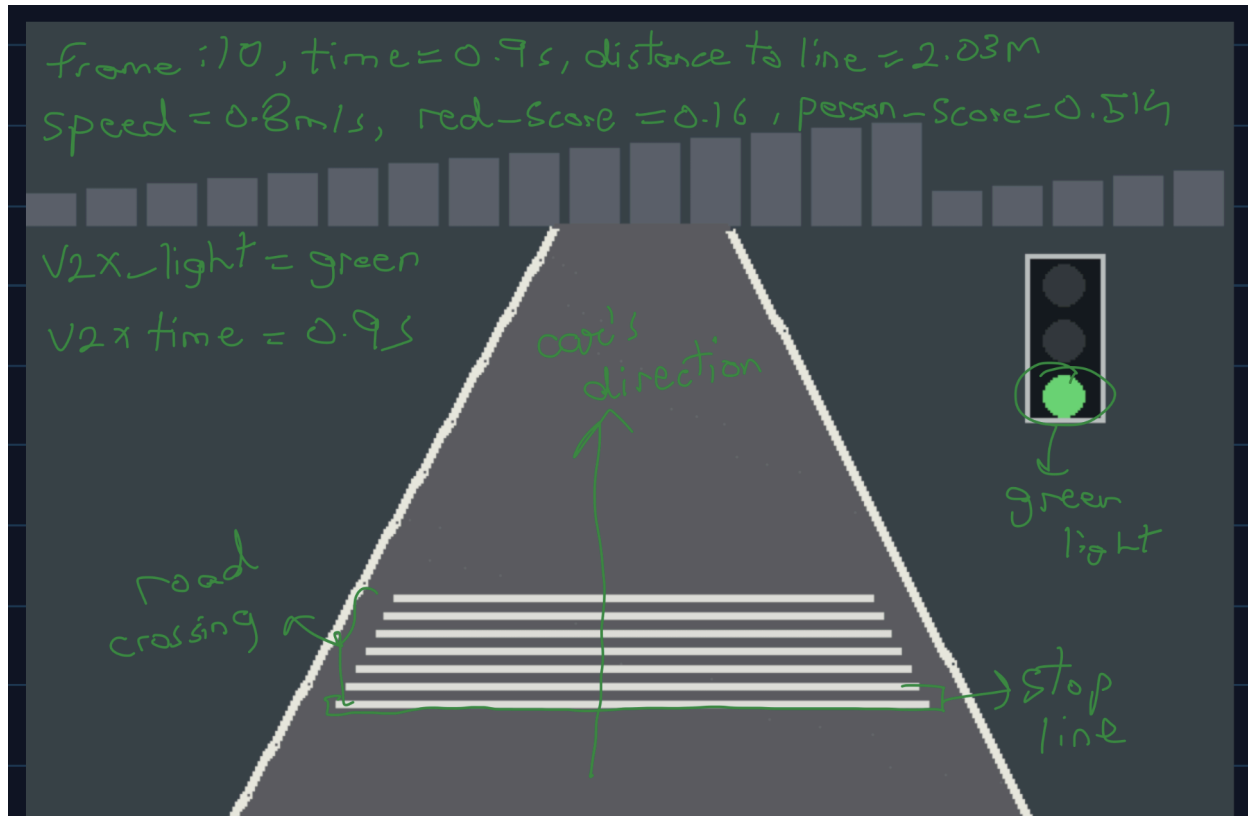
Measure the pedestrian's position and use the Question 1 road region instead of the supplied person score alone. Try a small belief filter for the light state.

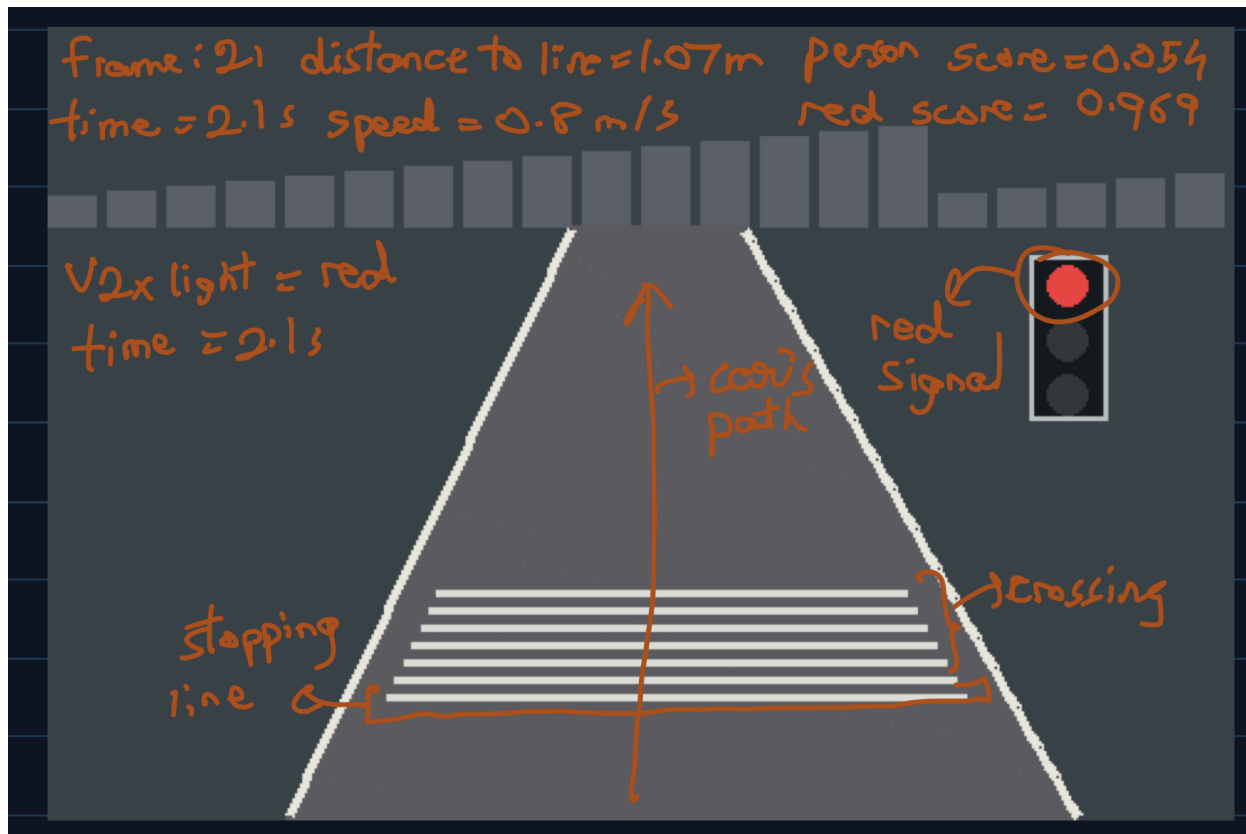
READING

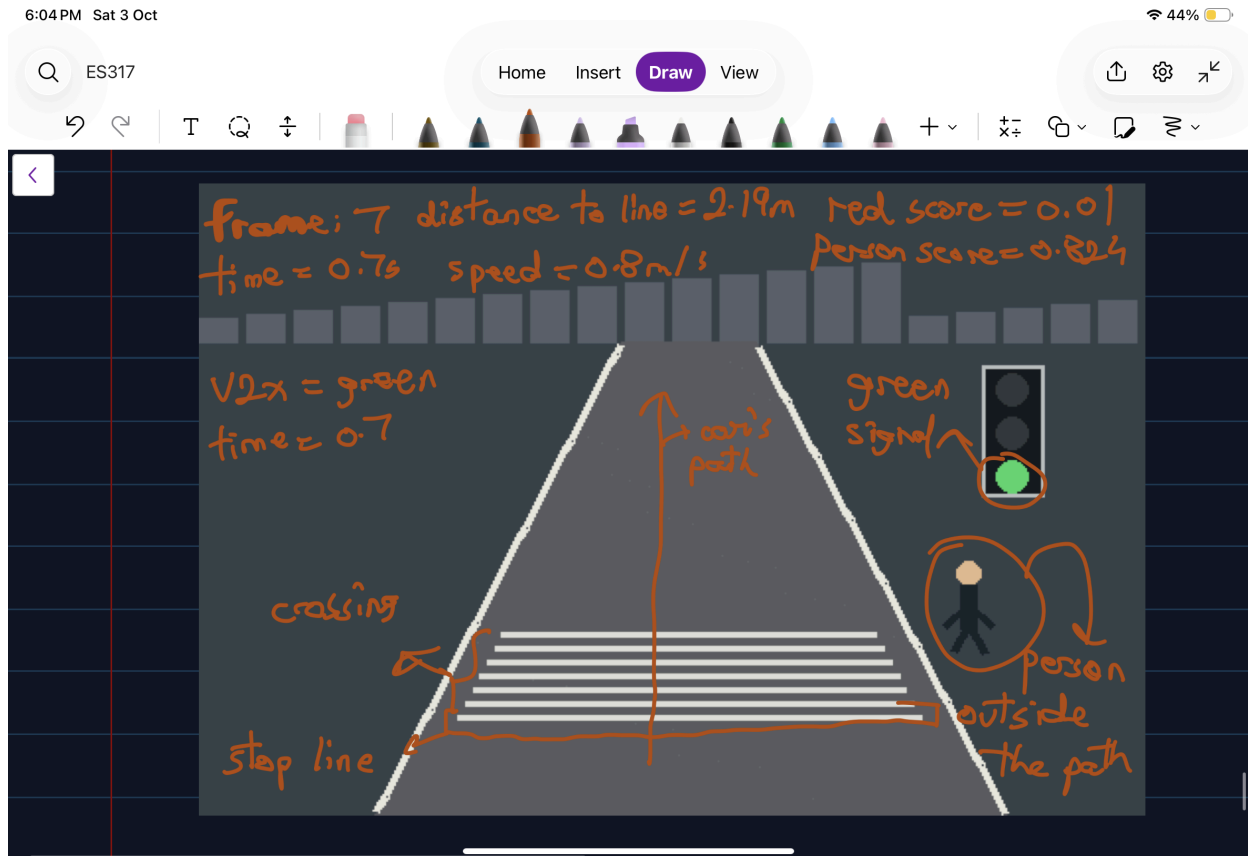
The [BFMC track and V2X rules](#) describe crossings and smart lights. OpenCV's [background-subtraction tutorial](#) is one route to a motion cue.

Answer:

a)









b) Baseline Decision Logic

⇒ Signal-Flow Diagram

- Due to inconsistency in the supplied person-score, we will use image measurement as the primary tool for detecting a person and person-score as secondary

Primary :

Legs of the human are detected
(black colour)



Compulsory STOP if legs have come
on the crossing



SLOW if person detected but
not yet on the crossing

Secondary :

IF person-score ≥ 0.8 , Compulsory
STOP



IF person-score ≥ 0.5 & < 0.8 ,
then SLOW (unresolved evidence)

- Red-score is consistent, doesn't require image measurement



IF red-score ≥ 0.8 , compulsory STOP



IF red-score ≥ 0.5 & < 0.8 , SLOW

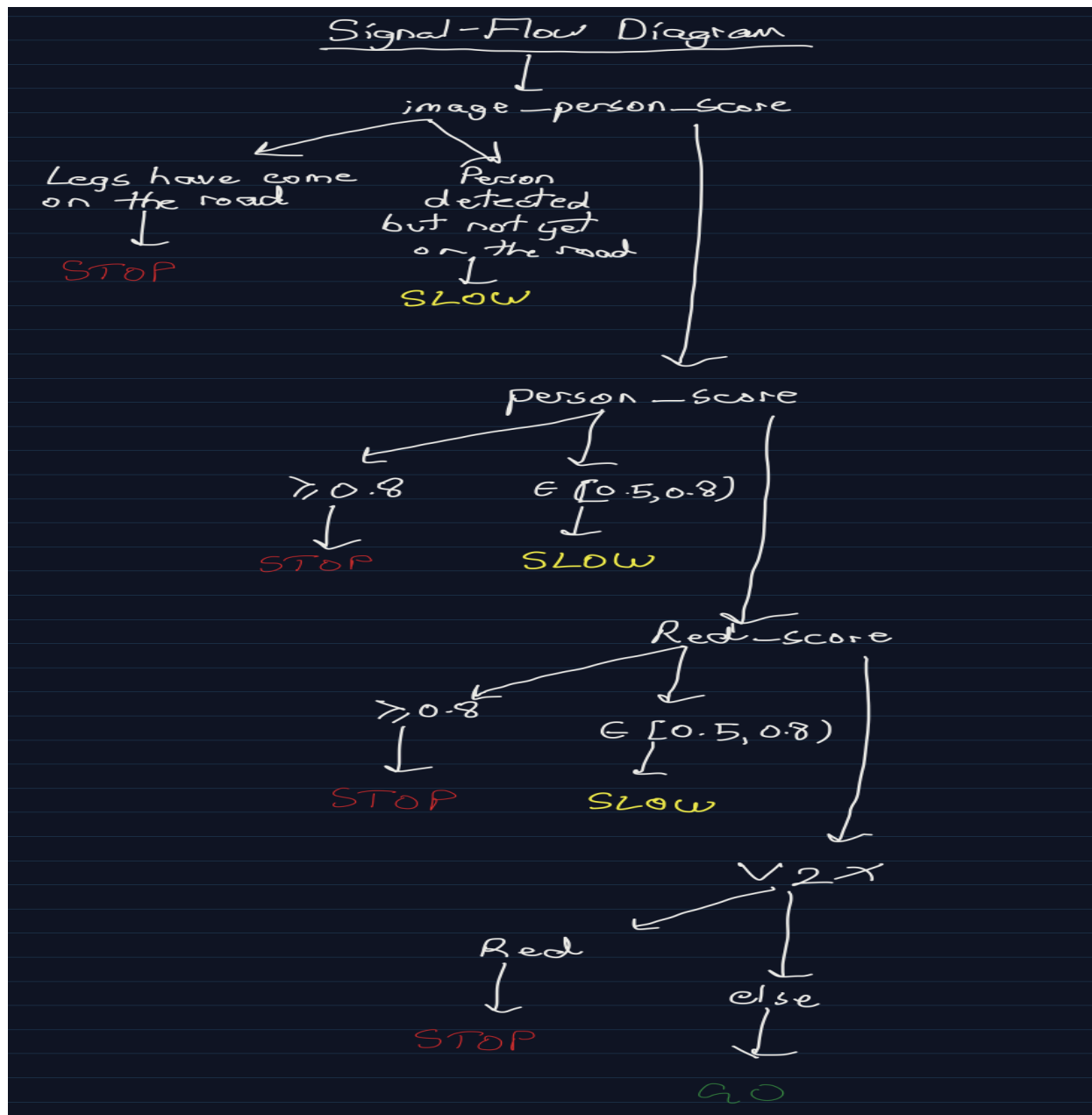
- V2X

↳ IF V2X is RED, compulsory STOP

All ELSE GO



Signal Flow Diagram



STOP>SLOW>GO (except for the exception of image person score, where slow is above person score red)



Output of Baseline Code:

Clear Green:

COMPACT TIMELINE

Frame 00 (t=0.00s) → GO

Frame 01 (t=0.10s) → GO

Frame 02 (t=0.20s) → GO

Frame 03 (t=0.30s) → GO

Frame 04 (t=0.40s) → GO

Frame 05 (t=0.50s) → GO

Frame 06 (t=0.60s) → SLOW

Frame 07 (t=0.70s) → SLOW

Frame 08 (t=0.80s) → GO

Frame 09 (t=0.90s) → SLOW

Frame 10 (t=1.00s) → GO

Frame 11 (t=1.10s) → GO

Frame 12 (t=1.20s) → GO

Frame 13 (t=1.30s) → GO

Frame 14 (t=1.40s) → GO

Frame 15 (t=1.50s) → GO

Frame 16 (t=1.60s) → GO



Frame 17 ($t=1.70s$) → GO

Frame 18 ($t=1.80s$) → GO

Frame 19 ($t=1.90s$) → GO

Frame 20 ($t=2.00s$) → GO

Frame 21 ($t=2.10s$) → GO

Frame 22 ($t=2.20s$) → GO

Frame 23 ($t=2.30s$) → GO

Late Red

COMPACT TIMELINE

Frame 00 ($t=0.00s$) → GO

Frame 01 ($t=0.10s$) → SLOW

Frame 02 ($t=0.20s$) → GO

Frame 03 ($t=0.30s$) → GO

Frame 04 ($t=0.40s$) → GO

Frame 05 ($t=0.50s$) → GO

Frame 06 ($t=0.60s$) → GO

Frame 07 ($t=0.70s$) → GO

Frame 08 ($t=0.80s$) → GO

Frame 09 ($t=0.90s$) → STOP



Frame 10 ($t=1.00s$) → STOP

Frame 11 ($t=1.10s$) → STOP

Frame 12 ($t=1.20s$) → STOP

Frame 13 ($t=1.30s$) → STOP

Frame 14 ($t=1.40s$) → STOP

Frame 15 ($t=1.50s$) → STOP

Frame 16 ($t=1.60s$) → STOP

Frame 17 ($t=1.70s$) → STOP

Frame 18 ($t=1.80s$) → STOP

Frame 19 ($t=1.90s$) → STOP

Frame 20 ($t=2.00s$) → STOP

Frame 21 ($t=2.10s$) → STOP

Frame 22 ($t=2.20s$) → STOP

Frame 23 ($t=2.30s$) → STOP

Person Crossing

COMPACT TIMELINE

Frame 00 ($t=0.00s$) → GO

Frame 01 ($t=0.10s$) → GO

Frame 02 ($t=0.20s$) → GO

Frame 03 ($t=0.30s$) → GO

Frame 04 ($t=0.40s$) → GO



Frame 05 ($t=0.50s$) → GO

Frame 06 ($t=0.60s$) → GO

Frame 07 ($t=0.70s$) → SLOW

Frame 08 ($t=0.80s$) → SLOW

Frame 09 ($t=0.90s$) → SLOW

Frame 10 ($t=1.00s$) → STOP

Frame 11 ($t=1.10s$) → STOP

Frame 12 ($t=1.20s$) → STOP

Frame 13 ($t=1.30s$) → STOP

Frame 14 ($t=1.40s$) → STOP

Frame 15 ($t=1.50s$) → STOP

Frame 16 ($t=1.60s$) → STOP

Frame 17 ($t=1.70s$) → STOP

Frame 18 ($t=1.80s$) → STOP

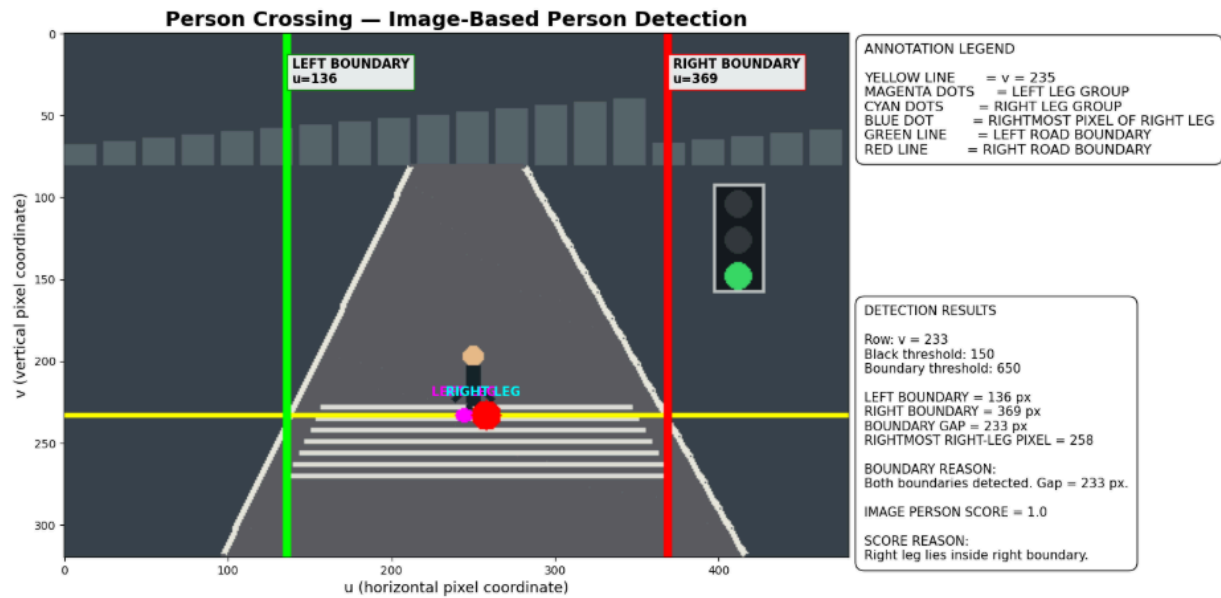
Frame 19 ($t=1.90s$) → STOP

Frame 20 ($t=2.00s$) → STOP

Frame 21 ($t=2.10s$) → STOP

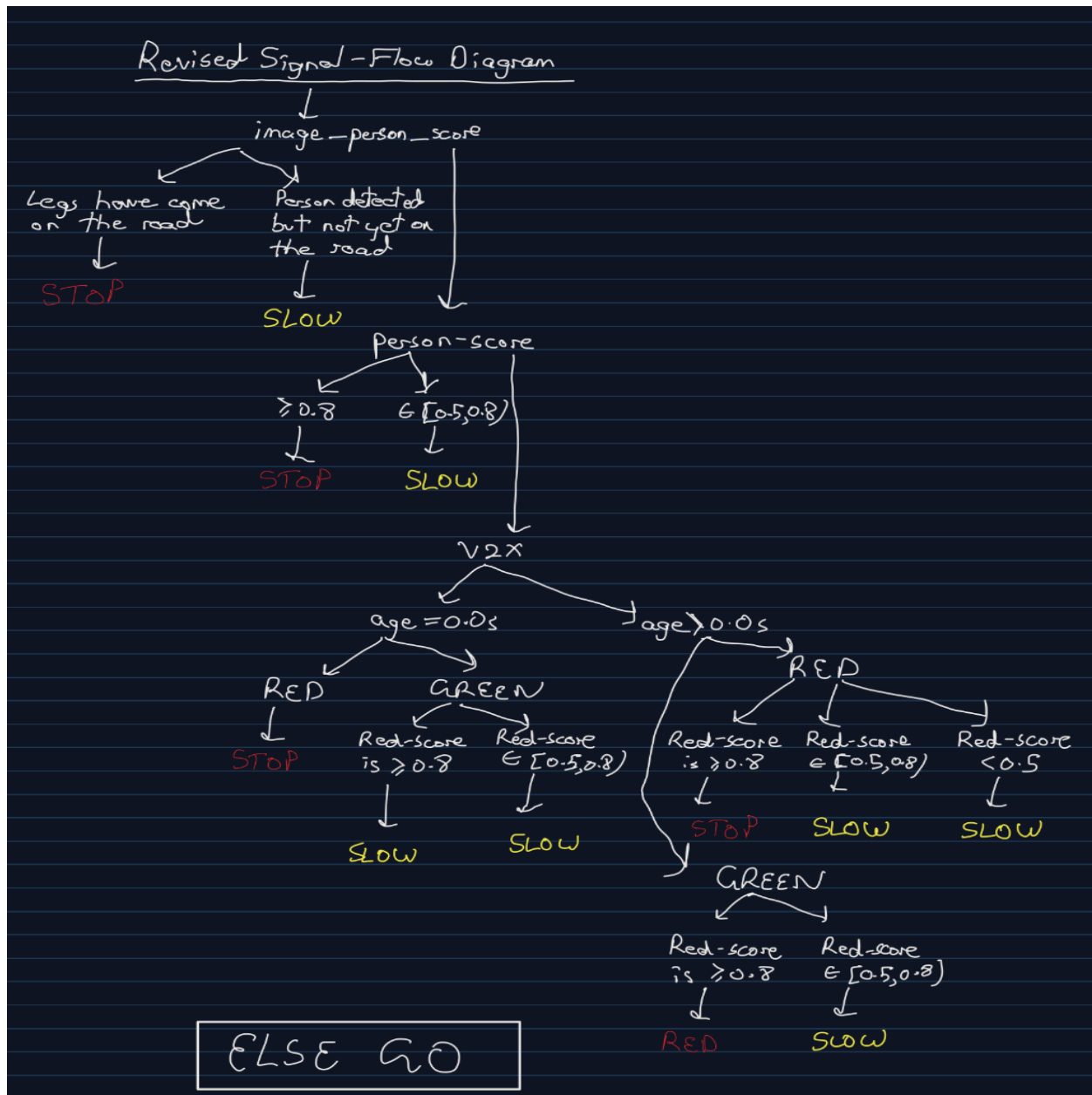
Frame 22 ($t=2.20s$) → STOP

Frame 23 ($t=2.30s$) → STOP



c)

Revised Signal Flow Diagram:



STOP>SLOW>GO (except for the exception of image person score, where slow is above person score red)



Clear Green:

Frame	Time (s)	Baseline	Revised	Change
00	0.00	GO	GO	—
01	0.10	GO	GO	—
02	0.20	GO	GO	—
03	0.30	GO	GO	—
04	0.40	GO	GO	—
05	0.50	GO	GO	—
06	0.60	SLOW	SLOW	—
07	0.70	SLOW	SLOW	—



UNMESH MASHRUWALA
Innovation Cell
IIT BOMBAY

08	0.80	GO	GO	—
09	0.90	SLOW	SLOW	—
10	1.00	GO	GO	—
11	1.10	GO	GO	—
12	1.20	GO	GO	—
13	1.30	GO	GO	—
14	1.40	GO	GO	—
15	1.50	GO	GO	—
16	1.60	GO	GO	—
17	1.70	GO	GO	—





UNMESH MASHRUWALA
Innovation Cell
IIT BOMBAY

18	1.80	GO	GO	—
19	1.90	GO	GO	—
20	2.00	GO	GO	—
21	2.10	GO	GO	—
22	2.20	GO	GO	—
23	2.30	GO	GO	—

Late Red:

Frame	Time (s)	Baseline	Revised	Change
00	0.00	GO	GO	—



UNMESH MASHRUWALA
Innovation Cell
IIT BOMBAY

01	0.10	SLOW	SLOW	—
02	0.20	GO	GO	—
03	0.30	GO	GO	—
04	0.40	GO	GO	—
05	0.50	GO	GO	—
06	0.60	GO	GO	—
07	0.70	GO	GO	—
08	0.80	GO	GO	—
09	0.90	STOP	STOP	—
10	1.00	STOP	STOP	—





UNMESH MASHRUWALA
Innovation Cell
IIT BOMBAY

11	1.10	STOP	STOP	—
12	1.20	STOP	STOP	—
13	1.30	STOP	STOP	—
14	1.40	STOP	STOP	—
15	1.50	STOP	STOP	—
16	1.60	STOP	STOP	—
17	1.70	STOP	STOP	—
18	1.80	STOP	STOP	—
19	1.90	STOP	STOP	—
20	2.00	STOP	STOP	—





UNMESH MASHRUWALA
Innovation Cell
IIT BOMBAY

21	2.10	STOP	STOP	—
22	2.20	STOP	STOP	—
23	2.30	STOP	STOP	—

Person Crossing:

Frame	Time (s)	Baseline	Revised	Change
00	0.00	GO	GO	—
01	0.10	GO	GO	—
02	0.20	GO	GO	—
03	0.30	GO	GO	—
04	0.40	GO	GO	—



UNMESH MASHRUWALA
Innovation Cell
IIT BOMBAY

05	0.50	GO	GO	—
06	0.60	GO	GO	—
07	0.70	SLOW	SLOW	—
08	0.80	SLOW	SLOW	—
09	0.90	SLOW	SLOW	—
10	1.00	STOP	STOP	—
11	1.10	STOP	STOP	—
12	1.20	STOP	STOP	—
13	1.30	STOP	STOP	—
14	1.40	STOP	STOP	—





UNMESH MASHRUWALA
Innovation Cell
IIT BOMBAY

15	1.50	STOP	STOP	—
16	1.60	STOP	STOP	—
17	1.70	STOP	STOP	—
18	1.80	STOP	STOP	—
19	1.90	STOP	STOP	—
20	2.00	STOP	STOP	—
21	2.10	STOP	STOP	—
22	2.20	STOP	STOP	—
23	2.30	STOP	STOP	—



Overall Result:

Episode	Baseline first STOP	Revised first STOP	Improvement
Clear Green	None	None	—
Late Red	0.90 s	0.90 s	None
Person Crossing	1.00 s	1.00 s	None

d)

=====

=

CLEAR_GREEN — FIRST STOP TIMELINESS

=====

=

No STOP request in this episode.

=====

=

LATE_RED — FIRST STOP TIMELINESS

=====

First STOP frame : 9



Time : 0.90 s
Speed : 0.800 m/s
Distance to line : 2.030 m
Reaction distance : 0.120 m
Braking distance : 0.400 m
Required stopping dist.: 0.520 m
Comfortable stopping : YES

The available distance is sufficient for comfortable stopping.

=====

=

PERSON_CROSSING — FIRST STOP TIMELINESS

=====

=

First STOP frame : 10
Time : 1.00 s
Speed : 0.800 m/s
Distance to line : 1.950 m
Reaction distance : 0.120 m
Braking distance : 0.400 m
Required stopping dist.: 0.520 m
Comfortable stopping : YES

The available distance is sufficient for comfortable stopping.



Clear Green

There is **no STOP request** in the episode. The system briefly enters SLOW at frames 06-07 and 09 because of the available uncertain evidence, but since no STOP is ultimately requested, stopping-distance analysis is not applicable.

Late Red

- **First SLOW:** Frame 01 at **0.10 s**
- **First STOP:** Frame 09 at **0.90 s**
- The initial SLOW is triggered by **moderate evidence**, before the evidence becomes strong enough to satisfy a STOP condition.
- The later STOP occurs when the evidence becomes sufficiently strong, particularly the **red-light evidence / red_score** under the decision thresholds.

Person Crossing

- **First SLOW:** Frame 07 at **0.70 s**
- **First STOP:** Frame 10 at **1.00 s**
- The initial SLOW represents **uncertain/moderate pedestrian evidence**.
- The decision changes to STOP when the pedestrian evidence becomes sufficiently strong to cross the STOP threshold.

e)

=====

AVERAGE PERFORMANCE OVER 20 TRIALS

=====



Episode	Baseline Missed STOP	Revised Missed STOP	Baseline Unnecessary STOP	Revised Unnecessary STOP	Baseline Response Delay (s)	Revised Response Delay (s)
Clear Green	0.0	0.0	0.5	0.5	N/A	N/A
Late Red	0.0	0.0	0.0	0.0	0.0	0.0
Person Crossing	0.0	0.0	0.0	0.0	0.0	0.0



=====

=====

TOTAL PERFORMANCE OVER 20 TRIALS

=====

=====

Baseline missed STOP frames : 0

Revised missed STOP frames : 0

Baseline unnecessary STOP frames : 10

Revised unnecessary STOP frames : 10



=====

=====

FALSE ALARM — ORIGINAL BASELINE

=====

=====

No false alarm found in the original baseline.

=====

=====

DANGEROUS DELAY — ORIGINAL BASELINE

=====

=====

No dangerous delay found in the original baseline.

Question 3: The road mask that lied *(Bonus)*

A miniature city car is about to enter a busy street. It can follow the road only if it can tell road from sidewalk and notice people and vehicles near its path. A pretrained neural network paints a class label on every pixel. Its result looks impressive on a clear image, but a dark frame or a blurred edge can make a sidewalk look like road. Your job is to turn a segmentation model into an inspectable road-perception component, then find a case where its confident-looking picture would mislead the car. This is an experiment about using and questioning deep learning, not an exercise in training a network from scratch.

DATA AND MODEL

Download the [Cityscapes8 image and mask archive](https://umiciitb.org/) (about 18 MB). It contains eight real urban images and their pixelwise reference masks, with four images in train and four in val. Use the



images for inference. Keep the masks out of your prediction code; use them only to score and diagnose finished predictions. These are full-size streets, not BFMC track images, so do not assume the model's behaviour transfers unchanged to a miniature city. The checkpoint was trained for Cityscapes, and this tiny sample is not a held-out model benchmark. Treat it as eight concrete failure investigations, not evidence that a model is safe on every road.

The recommended starting model is [NVIDIA's Cityscapes-pretrained SegFormer-B0](#), used with the [Transformers SegFormer image processor](#). It can run inference on a CPU; no fine-tuning is required. You may use another lightweight model trained for the Cityscapes classes if you record its exact name and weights. The model predicts classes including road, sidewalk, person, rider, car, and bicycle. Use the model configuration to read its class names. Inspect the dataset's mask encoding and the [Cityscapes8 label mapping](#) before scoring: raw mask pixel values need not equal the model's class indices. Ignore pixels labelled void or ignore. If you resize a mask, use nearest-neighbour interpolation so class IDs are not blended.

For a class such as road, intersection-over-union compares predicted and reference pixel sets:

$$IoU = \frac{|P \cap G|}{|P \cup G|}$$

where P is the set of pixels predicted as that class and G is the reference set. Also count false-road pixels: non-road reference pixels that your system calls road. A high overall accuracy can hide precisely this dangerous error if most of the image is sky or buildings.

YOUR WORK

- a. Look at the street first.** Open all eight images before running a model. Choose two that would make different demands on a miniature car, such as a broad clear road and a crowded or visually ambiguous edge. Mark the road, the nearby sidewalk boundary, and any person or vehicle that could matter. In a few sentences, say what you would want a perception system to report and what you would not trust it to decide from one image alone.
- b. Run and decode the network.** Run the pretrained model on all eight images with fixed settings and save class-coloured overlays beside the originals. Record the model identifier, input size, preprocessing, and inference settings. Inspect one raw output and explain how you converted its lower-resolution class scores to a mask at the original image size. Check the class-ID mapping on one image before computing scores. Report road IoU and false-road pixels for each image, plus a short summary for the four val images. The *train/val* split is only for keeping your rule design separate from its check; neither subset measures generalization of the pretrained model.



- c. Make a useful output, not just a colourful picture. Define a small image-based status for the car: *ROAD_VISIBLE*, *UNCERTAIN*, or *ROAD_NOT_VISIBLE*. For example, inspect the lower central part of the predicted road mask, its continuity, nearby sidewalk pixels, and the model's road scores. You choose the policy, but write it as a reproducible function of the model output and explain each threshold. Show its status and reason on every image. This is an image-evidence status, not permission to drive; a single image does not provide distance, speed, or right of way.
- d. **Stress the camera.** Make at least two controlled versions of every image, such as darker exposure and mild blur. Preserve the original reference mask for brightness and blur changes, keep the same transformation settings for all images, and record a random seed if any step is random. Run the unchanged model and status rule on these versions. Compare road IoU, false-road pixels, and status changes with the originals. Pick one perturbation that exposes a genuine weakness; show original image, altered image, reference road mask, and both predicted masks side by side. Do not claim that eight images estimate real-world reliability.
- e. **Design one repair and a failure card.** Make one justified intervention without retraining: for example, withhold *ROAD_VISIBLE* when the road score is low, check disagreement between two image regions, or use a simple image enhancement before inference. Fix its settings using only the four train images and your stated reasoning, then rerun all four val images and their altered versions. Report what improved, what became worse, and at least one case the repair still misses. End with a one-page failure card: the image, model output, reference, mistaken status, probable cause, repair, and what evidence a real car would need before acting.

Do not call a softmax score a calibrated probability unless you test calibration. A model can draw a plausible mask and still put the wrong class on the pixels that matter most. We value a defensible *UNCERTAIN* result more than an unsupported claim that the street is clear.

DELIVERABLES

Eight original-image segmentation overlays; per-image road IoU and false-road counts; the status rule and an eight-image status table; paired results for two image perturbations; a four-image val comparison before and after your repair; and the one-page failure card. Include runnable inference code, model identifier, thresholds, and exact perturbation settings.



FURTHER EXPLORATION

Compare a second pretrained Cityscapes model under the same settings, or add a small manually defined boundary-risk measure. Do not train on the four evaluation images and present their score as a held-out result.

READING

The [Cityscapes8 dataset guide](#) explains the sample and its limits. The [SegFormer documentation](#) explains preprocessing and resizing model outputs. The [model page](#) gives a starting checkpoint. Read the [label mapping](#) before evaluation.

Question 4: Which part of the track are we on? (Bonus)

The race car has a short map with eight named places. At two of them, the laser scans are so similar that one observation may point to the wrong place. Odometry, the estimate of whether the car stayed or moved, is also imperfect. A useful localizer should carry several possible places forward, strengthen or weaken them when a new scan arrives, and say when it cannot distinguish them. You will first try a single-scan guess, then build a small belief filter. No SLAM package or robotics course is needed; the route, candidate scans, and update rule are supplied.

DATA AND COORDINATES

The route has eight candidate places numbered 0 to 7. *localization/route.png* sketches it, and *localization/route_graph.csv* tells you where a stay or move action leads from each place. A move from place 7 wraps to place 0. The input file *localization/reference_scans.csv* is your map: it gives the expected 31-beam laser fingerprint at each place. It is called “reference” because it is a stored map scan, not because it reveals the answer to the route.

The input file *localization/observed_scans.csv* contains 12 rows, one per time step. Each row has an odometry message and 31 noisy ranges *r_00* through *r_30*. The beams run from -75° to $+75^\circ$ in 5° steps, and ranges are in metres. Match beams by column name when comparing an observation with a map fingerprint. The file *localization/route_reference.csv* reveals the true place at each step; keep it out of the localizer and use it only for evaluation.

A place is a discrete location on this route, not a continuous (x,y) pose. The 31 numbers in a scan may resemble more than one place. A belief assigns a probability to each of the eight possibilities, so all eight values sum to one. The challenge is to combine what the scan resembles with where the car could have moved since the previous step. This is a small, inspectable model, not full scan matching or SLAM.



YOUR WORK

- Find the look-alikes.** Plot all eight map fingerprints on labelled axes. Name a pair of places whose scans are easy to confuse and identify the beam angles that make them similar or different. Use this observation to explain why “pick the closest scan” may sometimes fail.
- Make the single-scan baseline.** For each of the 12 observed scans, choose the closest map fingerprint using a distance measure you define. A mean absolute or squared difference is a valid start. State how you treat missing or unusually large beam errors; one corrupted range should not decide the entire place unless you can justify it. Plot the predicted place against time step and keep these guesses for comparison.
- Carry uncertainty through time.** Before reading the first scan, assign probability 0.5 to place 0 and 0.5 to place 4, with zero elsewhere. At time step 0, update this prior using the first scan; ignore the row's stay message because there is no previous step to move from. For each later step, use its odometry message to predict where the car might be, then use its scan to update that prediction. Let $b_t(j)$ be the probability of place j after the scan at time t and let $\bar{b}_t(j)$ be the prediction before that scan:

$$\bar{b}_t(j) = \sum_i P(j | i, u_t) b_{t-1}(i), \quad b_t(j) = \frac{P(z_t | j) \bar{b}_t(j)}{\sum_k P(z_t | k) \bar{b}_t(k)}.$$

Here u_t is the odometry message, z_t is the observed scan, $P(j | i, u_t)$ is the chance of moving from place i to j , and $P(z_t | j)$ measures how well the scan fits place j . For move, start with probability 0.8 of following the graph's move edge and 0.2 of staying. For stay, start with 0.9 for staying and 0.1 for moving. Choose and explain a scan-fit rule, such as giving smaller scan error a higher likelihood. Normalize after every update and check that the eight probabilities sum to one. You may revise the starting numbers after testing, but report both versions.

- Show when the filter is unsure.** Plot a heatmap with time steps on one axis, places 0 to 7 on the other, and colour showing $b_t(j)$. Define an uncertainty rule, such as a low maximum probability or two nearly equal leading places. Compare the most likely place at each step with the single-scan baseline and, only after both are fixed, with *route_reference.csv*. Find the first step where the single-scan guess and running belief disagree. Use the scan, motion message, and earlier belief to explain which answer you trust and why.



- e. **Test a mistaken assumption.** Increase the scan noise your likelihood assumes, or corrupt selected observed beams with a stated seed. Rerun the same route and compare place accuracy, the mean probability given to the true place, and the number of uncertain steps. If the filter becomes confidently wrong, trace the error to a scan-fit or motion assumption. Finally, propose one extra observation or short movement that would distinguish the most confusing places. Identify the beams or future map place that make your proposal useful.

The posterior is a distribution over possibilities, not a guarantee. If it becomes sharply concentrated on the wrong place, say which assumption caused the overconfidence.

DELIVERABLES

A labelled plot of the eight map fingerprints; the single-scan predictions; a belief heatmap; baseline-versus-filter accuracy and true-place probability; your uncertainty rule and its output; a diagnosis of one disagreement or overconfident error; and the extra observation or movement you propose.

FURTHER EXPLORATION

Try a robust scan likelihood, different transition probabilities, or a small particle representation. Evaluate against the same 12-step route.

READING

The [University of Washington localization assignment](https://umiciitb.org/) illustrates motion and observation models.



MOTION PLANNING

Q1 is mandatory. Q2 is recommended. Q3 is BFMC-specific and optional. Budget about 6 to 9 focused hours for Q1.

All implementation must be done in a Jupyter notebook/Google Colab. You may use relevant Python libraries such as NumPy and Matplotlib, as needed.

Question 1: Planning for a vehicle that carries people *(Mandatory)*

A 10×10 grid with plain A is a fine way to learn path planning. It is not a way to move a laden, passenger-carrying vehicle down a road, because it ignores the one thing that matters most: the vehicle has a heading, and it cannot turn on the spot.*

Part A: Fundamentals, applied (research)

- Define the configuration space for the buggy. How many dimensions, and what are they? How does this differ from the C-space of a point robot, and why does that difference matter for a planner?
- The buggy has a minimum turning radius set by its wheelbase and max steer angle, compute it from the platform spec. Now: take a path that plain 8-connected grid A* would return between two points with a tight turn between them. Show, concretely, why the buggy cannot execute it.
- Research Hybrid A*. In your own words, explain how Hybrid A* differs from normal grid-based A* and why it is more suitable for a car-like vehicle such as the buggy.

Part B: Build it

Implement the following entirely in a Jupyter Notebook (.ipynb).

- Implement a normal 2D grid-based A* planner. Your implementation should allow the user to define the grid size, start position, goal position, and obstacle locations. Visualise the explored nodes and the final path found by A*.
- Test the planner on a simple road-like environment containing at least one bend and 3–4 static obstacles. Visualise the resulting path and explored states.



- f. Check the path from (e) against the minimum turning radius you computed in (b). Mark every turn the buggy could not execute and explain what the planner would have to change to fix it. Optional extension: implement Hybrid A* on the same environment and compare the two paths.

Part C: Planner Design Under Competing Constraints

Suppose two collision-free paths connect the same start and goal:

- Path 1 is shorter, but requires higher curvature and passes closer to obstacles.
- Path 2 is longer, smoother, and maintains a larger clearance from obstacles.

Design a cost function that would allow the buggy to choose between them.

Your answer should discuss how you would combine at least the following terms:

- path length,
- curvature or steering effort,
- obstacle clearance,
- and change in steering between consecutive steps.

Explain what could go wrong if any one of these terms is weighted too heavily, and how the preferred path might change if the buggy were moving at 5 km/h versus 20 km/h.

Answer:

Part A:

a)

Configuration Space: The buggy is represented by the configuration $q = (x, y, \theta)$, where x and y describe its position in the environment and θ describes its heading. Hence, its configuration space is three-dimensional.

In contrast, a point robot can be represented by only (x, y) , since its orientation and vehicle geometry do not constrain its motion. For the buggy, orientation is essential because it is a car-like, nonholonomic vehicle and cannot move or rotate arbitrarily. Its steering angle is limited to $\pm 32^\circ$, and therefore its future position depends on both its current heading and steering input.



b)

For a **kinematic bicycle model**, the turning radius is

$$R = \frac{L}{\tan \delta}$$

where:

- R = turning radius
- L = wheelbase
- δ = front-wheel steering angle

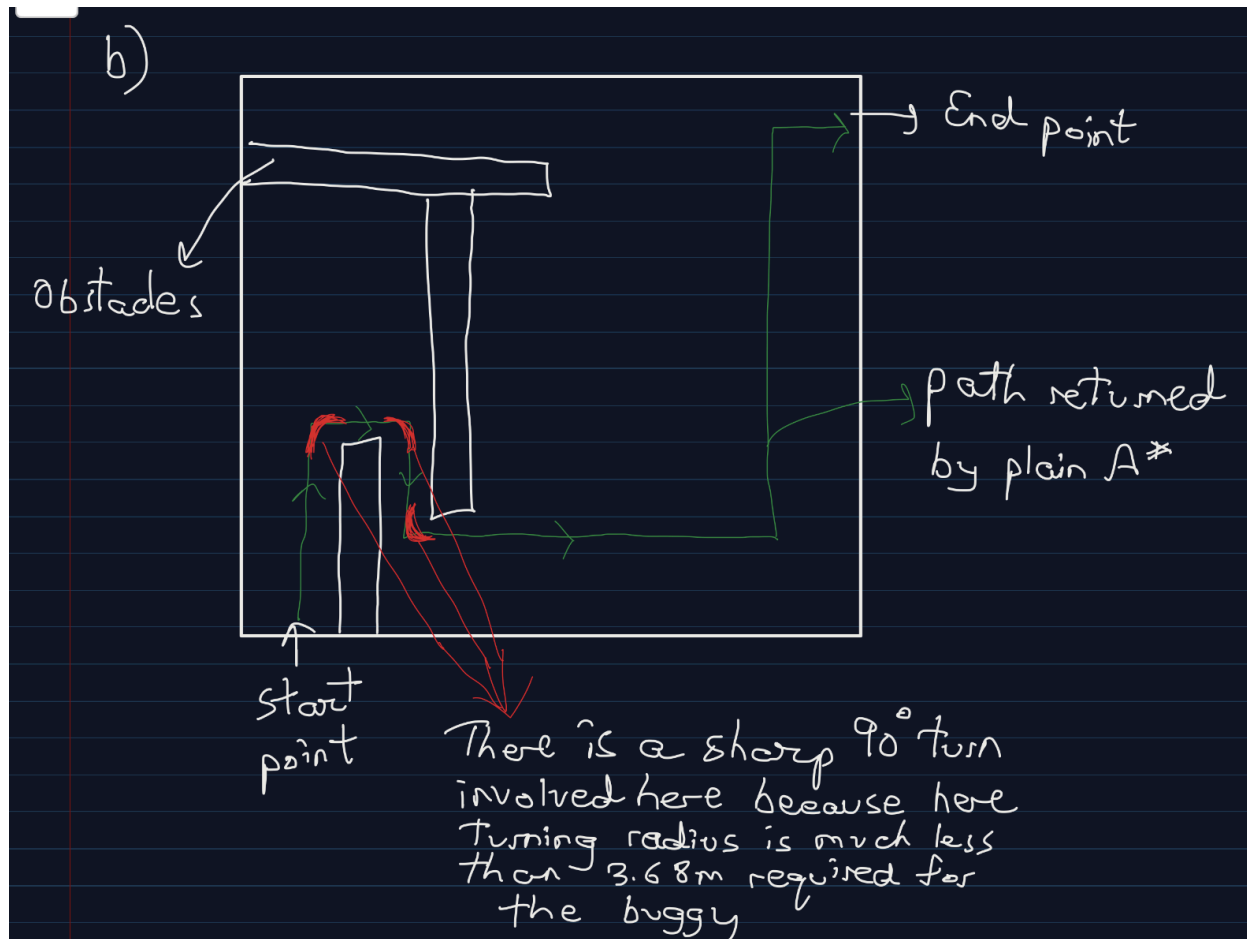
Therefore,

$$R_{\min} = \frac{2.30}{\tan 32^\circ}$$

$$\tan 32^\circ \approx 0.6249$$

so

$$\mathbf{R_{\min} = 3.68m}$$



c)

Hybrid A* uses 3 dimensions (x,y, theta), where theta is the direction in which the buggy is heading. This is important because, for deciding in which cell the buggy should go next, the heading of the buggy is very important, since a buggy is not a point object. Hybrid A* , not only gives the most optimized path but also the kinematically feasible trajectory where the buggy isn't required to make any sharp turn, and the planner accounts for the steering constraints of the buggy.



Question 2: Where is everyone else going? Trajectory prediction

(Recommended)

A planner is only as good as its guess about what everyone *else* on the road is about to do. The buggy shares its roads with pedestrians, cyclists, and other vehicles who never signal their intent, and getting this wrong is how a "safe" plan turns into a near-miss.

Part A: Understand the problem

- a. Suppose the past positions of an agent are given as $(x_1, y_1), (x_2, y_2), \dots, (x_t, y_t)$ and the task is to predict its future positions. Briefly explain why a constant-velocity model can work in some situations and where it can fail. What information available in the recent trajectory could be used to make a better prediction? You may research ideas such as constant acceleration, change in heading, weighted velocity estimates, polynomial fitting, Kalman filtering, or interaction-based models. You do not need to implement any learned model such as an LSTM or Transformer.

Part B: Build and compare

- b. Use a small public pedestrian or vehicle trajectory dataset. The ETH/UCY pedestrian dataset is recommended (a ready-to-use copy is in the [Trajectron++ repository](#) under experiments/pedestrians/raw), but any suitable public dataset may be used. Briefly describe what one trajectory in your chosen dataset represents and how the positions are recorded over time.
- c. Implement a constant-velocity baseline predictor using the recent observed positions of an agent. Use it to predict the trajectory approximately:
- 1 second into the future,
 - 3 seconds into the future.

Plot the observed trajectory, predicted trajectory, and actual future trajectory for a few examples.

- d. Design and implement one improved predictor of your choice.

Your method should use more information than simply assuming the current velocity remains constant. You may use ideas such as:

- acceleration,
- change in heading,
- recent velocities with different weights,



- curve fitting,
- Kalman filtering,
- nearby agents,
- or a combination of these.

You are free to choose the method, but you must clearly explain why you expect it to improve prediction. The aim is not to reproduce a research paper. The aim is to design, implement, and test a reasonable prediction method.

- Evaluate both on held-out trajectories using Average Displacement Error (ADE) and Final Displacement Error (FDE) at a couple of prediction horizons (e.g. 1 s and 3 s). Report the numbers and plot a few representative predicted-vs-actual trajectories, including at least one case where your better model clearly helps and one where it still fails badly.

Part C: Connect it back

- Two pedestrians may have almost identical past trajectories but make completely different decisions in the next few seconds: one may continue straight while the other may suddenly cross the road. What information is fundamentally missing if the prediction uses only past x, y positions?

Propose one additional source of information that the buggy could use to reduce this uncertainty. Explain how it could improve prediction and what new failure modes it might introduce.

Question 3: BFMC routing and rules on the scale track *(Optional, BFMC-specific)*

- Obtain the official BFMC track map (from the documentation's [Race Track page](#)) and convert it into a graph, using nodes for intersections and important decision points and edges for valid road segments. Encode lane direction and permitted manoeuvres, and visualise the resulting graph in a Jupyter Notebook.
- Place 8 virtual waypoints on the track. The vehicle may visit them in any order. Formulate this as a routing problem, identify the classical optimisation problem it resembles, and compute a waypoint order and route. Report the final route and estimated travel time.
- Extend the routing cost to account for delays or penalties at features such as pedestrian crossings, parking zones, and traffic-light intersections. Recompute the route and explain how the waypoint order or chosen path changes compared with the distance-only solution.



UNMESH MASHRUWALA
Innovation Cell
IIT BOMBAY

- d. In the Arena Fight, the localisation server is unavailable and other cars are present on the track. Explain how this affects the planner and what fallback strategy the vehicle should use when reliable global localisation is temporarily unavailable.

SEDRICA



SIMULATION

Q1 is mandatory. Q2 is optional. Budget about 15 to 20 focused hours for Q1; keep Part D small.

This subsystem is building the safety-critical scenario generation pipeline described in the link below. It is the most research-flavoured work on the team, and this question is a genuine miniature of the actual project.

Important: It is recommended to read the BridgeGen paper (or its summary) linked below before starting.

<https://arxiv.org/pdf/2311.10937>

Question 1: Design a safety-critical scenario generation pipeline *(Mandatory)*

This question is primarily a design exercise, grounded by a small working proof of concept: it is not asking you to build the full pipeline in a week. Read through everything before starting; Part D is deliberately the smallest part.

The problem, stated precisely. You are handed an arbitrary driving policy as a black box. You can run it in simulation and observe what it does, but you cannot see inside it.

Your job is to design a system that finds the traffic situations in which this policy is most likely to fail dangerously, *before* it is ever allowed near a real passenger.

This is genuinely an open, actively researched problem, and there is no single correct architecture, so we are grading how clearly you reason through the design choices, not whether you land on the one we would have picked.

Part A: Why is this hard?

- a. *In 150 words or less:* why can't you validate a driving policy just by running it around IITB campus (or on the real road) and counting incidents? What does this imply about how rare "dangerous" events are, and about how they must be found?



Part B: Design an architecture

Design (as a labelled block diagram + written justification for every block) a pipeline that takes an arbitrary policy as input and outputs a catalogue of dangerous, plausible scenarios.

Your design must explicitly address each of the following, and your answer to each should be a reasoned position, not a list of options.

b. Search space.

- How do you define the space of possible scenarios in the first place? What are its dimensions (road geometry, other agents' initial states and behaviours, weather, etc.)
- How do you decide which dimensions matter enough to be included without the space becoming unsearchably large?
- Is this space the same for every policy you might test, or does it need to change depending on what the policy is meant to do?

c. Danger Measurement.

- How does your architecture decide a scenario is "dangerous"?
- Name at least two metrics you would use and state one situation where you would expect a single metric to mislead you.

d. Plausibility.

An unconstrained search will happily find "failures" where the other vehicle behaves like a malicious lunatic and drives into you: a true collision, and a useless test, because no real driver would do that.

- How does your architecture prevent this from dominating your results? Name a specific mechanism (a constraint on the adversary's physically-realistic behaviour, a likelihood term grounded in real driving data, a rules-based filter).

e. Search strategy.

Open-ended question: How will you navigate the search space in order to arrive at the most dangerous scenarios while maintaining plausibility (Part d), and how will you use the danger measurement metrics (Part c) in your approach?

Describe your approach and how you arrived at it. List all sources used for whatever purpose, including AI.



f. Where does GenAI fit, if anywhere?

Read about at least one recent approach that uses an LLM or another generative model somewhere in a scenario-generation pipeline. Examples include an LLM proposing scenario descriptions from a natural-language prompt like "an aggressive cut-in near a bus stop", or a generative model producing plausible adversary trajectories conditioned on real driving data.

Decide, and argue, where (if anywhere) such a component belongs in *your* architecture.

Part C: Stress-test your own design

- g.** Pick the one component of your architecture you are least confident about, and write ~150 words on why, and what you would prototype first to find out if your design choice was actually right.

Part D: A small proof of concept (keep this small)

- h.** Build a minimal, deterministic 2D simulator (an ego vehicle with simple kinematics, one other agent, and a straight or gently curved road) plus a trivial rule-based ego policy. Keep it small: this is a proof of concept, not a polished tool.
- i.** Implement one criticality metric from (c) and run one simple search; even plain random sampling over 3-4 scenario parameters is fine, to find a handful of scenarios where your metric flags danger.
- j.** Show 2-3 of the scenarios you found, and connect them explicitly back to your Part B design: does what you actually observed building even this small a system change anything about your architecture?

Question 2: Go one level deeper on one part of the design (Optional)

Pick one of the following and go deeper than Q1 required.

(i) Compare two search strategies properly.

Extend your Part D proof of concept:

- Implement a second, smarter search strategy (Bayesian optimisation, CMA-ES, cross-entropy method, or something else) alongside your random baseline.
- Compare them on the number of distinct dangerous scenarios found vs. the number of simulations run, across at least 3 random seeds. Report whether the smarter search's scenarios are still plausible, or whether it found more "failures" and why.



UNMESH MASHRUWALA
Innovation Cell
IIT BOMBAY

(ii) Ground your scenarios in real driving data

Take a small public trajectory dataset (e.g. highD, NGSIM or INTERACTION; a few hundred trajectory segments is enough), compute your criticality metric(s) across real trajectories in the dataset and compare the distribution against the scenarios your Part D search found. Are your generated scenarios inside the real distribution, at its tail, or implausibly far outside it? What do you infer from this?





BUSINESS SUBDIVISION

Section B: Business Subsystem Assignment

MEDIA & WEB

Q1 is mandatory. Q2 is mandatory. Q3 is a bonus. Budget about 12 to 18 focused hours for Q1 and Q2 together.

Q B.1: Launch the campus shuttle *(Mandatory)*

When the buggy first drives autonomously on the IITB campus, that will be a story. Your job is to make sure it lands.

- a.** Build a launch communication plan covering the two weeks around the first public autonomous run. Which platforms, what content, on what schedule, aimed at which audience (students, faculty, alumni, industry, press). Be specific: a calendar, not a philosophy.
- b.** Produce the hero LinkedIn post: the visual and the copy, ready to publish.
- c.** Produce one Instagram reel storyboard (shot list with 6-10 frames, on-screen text, audio direction, duration per shot).
- d.** Handle the hard part: address safety without either dismissing it or alarming people. A section of your audience's first reaction will be "is this thing going to run someone over?" Write the two or three sentences that appear in your public communication about safety. This is the part of the question we care most about: get it wrong in either direction and the vehicle gets shut down.
- e.** Define how you will measure whether the launch worked. Name the metrics and the targets.

Use any tool you like for the visuals: Canva, Figma, Illustrator. Judged on clarity of communication first, aesthetics second.



Q B.2: The website is where the campaign lands *(Mandatory)*

Every post in Q B.1 sends someone somewhere. That somewhere is our website and it has to be worth the click.

- a. Explore the UMIC SeDriCa website (<https://umiciitb.org/>) thoroughly. Propose at least six concrete improvements to its design, usability or functionality. For each: what you would change, why it is needed and how you would implement it. Include screenshots of the current state alongside wireframes or mockups of your proposal.
- b. Implement at least two of them: a polished mockup, an *HTML/CSS/JS* snippet or a working prototype. Deploy it somewhere (Vercel, Netlify, GitHub Pages) and give us the link.
- c. Connect the two halves of this subsystem: pick one campaign from your Q B.1 plan and describe what a visitor arriving from it should land on. Does that page exist today? If not, sketch or build it. If it does, say what you would change about it and why.

Q B.3: Past work *(Bonus)*

Share one or two things you have already made: a website, a social account you ran, a campaign, a design portfolio. For each, 200-300 words on what it was, what you were trying to achieve and, most usefully, the hardest problem you hit and how you solved it. Live links wherever they exist.

Appendix (required in every submission)

Include at the end of your report:

1. **References:** everything you read, per question.
2. **Decision log:** the non-obvious choices you made and why.
3. **Failure log:** what did not work, what you could not finish, what you would do with another week.
4. **AI usage note:** Prompts you used while using the AI tools substantially, if you did.

END OF ASSIGNMENT